TLV Cash Register Interface Specification

Product name	TLV Cash Register Interface
Product number	-
Title	TLV Cash Register Interface Specification
Author	Robert Demming
Version	v1.0.9
Status	Release
Classification	Confidential
Info	Saved as "*TLV Cash Register Interface Specification.docx"*, on 2018-03-27 14:10

Copyright Statement © 2016-2017 Chess pT International B.V

All rights reserved. Disclosure to third parties of this document or any part thereof, or the use of any information contained therein for purposes other than provided for by this document, is not permitted, except with the prior and express written permission of Chess payment Technology International B.V.

Document history

Date	Author	Status	Remarks
2016-01-14	rdem	Preview	First version for review.
2016-01-20	rdem	Release	First release
2016-11-02	rdem	Release	Minor additions to transaction type description.
2017-01-04	rdem	Release	Made bit number consistent throughout the document. Bits [8…1] instead of [7…0].
2017-04-25	rdem	Release	Fixed error in *HelloRequest* example.
2017-05-09	rdem	Release	Changed bit numbering to [8…1] in Table 3 and [16…1] in Table 13 to make bit numbering consistent.
2017-07-27	rdem	Release	v1.0.4: • Added “intermediate result” and “cancel” messages. • Added transaction stages *CardDetected* and *WaitingForIntermediateResult*. • Added additional transaction types.
2017-08-29	rdem	Release	v1.0.5: • Added “last transaction ticket” and “last transaction status” messages.
2017-09-15	rdem	Release	v1.0.6: • The “Intermediate result” now has a mode where the ECR does not have to reply with an *IntermediateResultResponse*.
2017-10-03	rdem	Release	v1.0.7: • Added “Version Information” messages. • Added terminal logical ID and serial number to *HelloRequest*.
2017-10-26	rdem	Release	v1.0.8: • Added transaction mode, authorisation code and token to the transaction request message. • Added appendix for Luhn check algorithm. • Added *ParameterMissing* and *ParameterInvalid* result codes. • Renamed *TransactionReference* field to *MerchantTransactionReference*.

2018-03-20	rdem	Release	V1.0.9: • Added.*CardHolderLanguage* field to *TransactionRequest*. • Added.*CardBrandIdentifier* field to *TransactionResponse*. • Added *TotalsRequest* and *TotalsResponse* messages. • Added *AcquirerListRequest* and *AcquirerListResponse* messages. • Added *DoAcquirerDownloadRequest* and *DoAcquirerDownloadResponse* messages. • Added *PendingMessagesAcquirerListRequest* and *PendingMessagesAcquirerListResponse* messages. • Added *DoSendOrDeletePendingMessageRequest* and *DoSendOrDeletePendingMessageResponse* messages. • Added *SecuritySchemeListRequest* and *SecuritySchemeListResponse* messages. • Added *DoSecuritySchemeDownloadRequest* and *DoSecuritySchemeDownloadResponse* messages. • Added *TmsSessionRequest* and *TmsSessionResponse* messages. • Added *RebootAndClearCtapDatabaseRequest* and *RebootAndClearCtapDatabaseResponse* messages.

References

[1]	TLV BER encoding: https://en.wikipedia.org/wiki/X.690#BER_encoding	
[2]	C-TAP.220 - Data Dictionary	v10.0 March 2012
[3]	ISO 639-1 language codes: https://en.wikipedia.org/wiki/ISO_639-1	
[4]	ISO 4217 currency codes: https://en.wikipedia.org/wiki/ISO_4217	
[5]	Bit numbering: https://en.wikipedia.org/wiki/Bit_numbering	
[6]	Luhn check: https://en.wikipedia.org/wiki/Luhn_algorithm	

Table of Contents

1 Overview 9
1.1 Introduction 9
1.2 Prerequisites 9
2 Message and Data Format 10
2.1 What is Tag-Length-Value (TLV) format 10
2.1.1 Tag Encoding 10
2.1.2 Length Encoding 10
2.2 Primitive Data Elements 11
2.3 Constructed Data Elements 12
2.3.1 Group Field 12
2.3.2 Table Field 12
3 Messages and Message Structure 14
3.1 Overview 14
3.2 Basic Message Structure 15
3.3 Message Header 16
3.3.1 Protocol ID 17
3.3.2 Message Type 17
3.3.3 Terminal ID 17
3.3.4 ECR ID 17
3.3.5 Sequence Number 17
3.3.6 Encryption Parameters Group 18
3.4 Message Body 18
3.5 Message Footer 18
3.5.1 MDC 18
4 Message Definitions 19
4.1 Hello Request/Response 19
4.1.1 Encryption Parameters Group 19
4.1.2 Communication Parameters Group 20
4.1.3 Connection Parameters Group 20
4.1.4 Supported Languages Group 20
4.1.5 Language (to be used) 21
4.1.6 Terminal Logical ID 21
4.1.7 Terminal Serial Number 21
4.1.8 Result Code 21
4.2 Keep-alive Request and Keep-alive Response 22
4.2.1 Keep-alive Reason 22
4.2.2 Terminal State 23
4.2.3 Result Code 23
4.3 Bye Request/Response 24
4.3.1 Bye Reason 24
4.3.2 Result Code 25
4.4 Information Report 26
4.4.1 Transaction ID 26
4.4.2 Terminal State 26
4.4.3 Transaction Stage 26
4.4.4 Stage Message 27
4.4.5 Merchant Text 27
4.5 Transaction Request/Response 28
4.5.1 Transaction ID 28
4.5.2 Transaction Data Group 29
4.5.3 Print Data Group 32
4.5.4 Terminal State 32
4.5.5 Transaction Result 32
4.5.6 Card Brand Identifier 33
4.5.7 Ticket Data Table 33
4.5.8 Result Code 34
4.6 Intermediate Result Request/Response 36
4.6.1 Transaction Result 36

4.6.2 Amount 36
4.6.3 Intermediate Result Mode 37
4.7 Cancel Request/Response 38
4.7.1 Cancel Reason 38
4.7.2 Terminal State 38
4.7.3 Result Code 38
4.8 Last Transaction Ticket Request/Response 40
4.8.1 Transaction ID 40
4.8.2 Print Data Group 40
4.8.3 Terminal State 40
4.8.4 Ticket Data Table 41
4.8.5 Result Code 41
4.9 Last Transaction Status Request/Response 42
4.9.1 Transaction ID 42
4.9.2 Terminal State 42
4.9.3 Result Code 42
4.9.4 Transaction Information Group 43
4.10 Totals Request/Response 44
4.10.1 Terminal State 44
4.10.2 Totals Type 44
4.10.3 Print Data Group 45
4.10.4 Ticket Data Table 45
4.10.5 Result Code 45
4.11 Security Scheme List Request/Response 46
4.11.1 Security Scheme List 46
4.12 Security Scheme Download Request 47
4.12.1 Security Scheme Identifier 47
4.13 Acquirer List Request/Response 48
4.13.1 Acquirer List Group 48
4.14 Acquirer Download Request 49
4.14.1 Acquirer Identifier 49
4.15 Acquirer List Pending Messages Request/Response 50
4.15.1 Acquirer List Group 50
4.16 Send or Delete Pending Messages Request 51
4.16.1 Pending Messages Operation 51
4.16.2 Acquirer Identifier 51
4.17 TMS Session Request/Response 52
4.18 Reboot and Clear CTAP Database Request 53
4.18.1 Reboot and Clear Type 53
4.19 Version Information Request/Response 54
4.19.1 Application ID 54
4.19.2 Terminal Logical ID 54
4.19.3 Terminal Serial Number 55
4.19.4 Version Number 55
4.19.5 Version String 55
4.19.6 Extra Information 55
4.19.7 Result Code 56
A Implementation Checklist 57
B Enumerations 58
B.1 Message Types 58
B.2 Result Codes 60
B.3 Transaction Stage 61
C CRC-32 Algorithm 62
D Luhn Check Algorithm 64
D.1 Luhn Check Algorithm (C++) 64
D.2 Luhn Check Algorithm (C#) 65
E Example Messages 67
E.1 Hello Request 67
E.2 Hello Response 68
E.3 Transaction Request 68

E.4 Transaction Response 68
E.5 Bye Request 69
E.6 Bye Response 69

1 Overview

1.1 Introduction

The Chess pT cash register interface is a software interface that makes it possible to connect electronic cash registers (ECR) and vending machines with an Ingenico payment terminal provided by Chess pT.

This manual is intended for cash register and vending machine providers who want to connect their cash register or vending machine to Ingenico terminals. This manual describes the direct interface between the terminal and a cash register or vending machine using the TLV protocol. The interface consists of messages (requests and responses) that are sent between the cash register / vending machine and terminal.

In case of vending machines in combination with unattended terminals, the interface enables transactions by sending a transaction request and receiving the transaction results.

When used with a POS (attended) terminal and a cash register, the cash register can send payment requests to the terminal without the merchant needing to manually enter the amount payable on the terminal. This eliminates the chance of errors and speeds up the payment process. When the transaction is complete, the cash register will receive the payment result and payment receipts for integration in the cash register receipt.

The interface is the same for both cash registers and vending machines. In this document we use “cash register” but you could also read “vending machine”.

1.2 Prerequisites

This manual is intended for software developers.

The cash register can communicate with the terminal via Ethernet (TCP/IP), serial connection (RS-232) or USB.

When the terminal is used with Ethernet, the developer is assumed to be familiar with TCP/IP networking concepts.
Similarly, for use with a serial connection it is assumed the developer has knowledge of serial communication concepts. When connected via USB the terminal is seen as a COM port at the PC side. Thus from the software point of view a USB connection is the same as a serial connection.

The cash register connection is asynchronous in the sense that both parties can send data at their own decision and not only after a request. For example the terminal is sending keep-alive requests while it is waiting for a command from the cash register. Thus the software must divide sending and receiving messages in separate loops. Therefore knowledge of multi-threading concepts is required.

2 Message and Data Format

2.1 What is Tag-Length-Value (TLV) format

The cash register interface is based on the Tag-Length-Value (TLV) format in which data is preceded by an ID (the tag) and the length of the data that follows. This format enables skipping of data of which the tag is unknown to the parser and thus enables adding extra data in future versions without affecting existing users. Also the order in which TLVs occur is not important and can be in any order.

The tag-length encoding is based (but not the same) on BER encoding (see [1]). This encoding describes how the tag and length parts of the TLV are encoded with a variable number of bytes.

Further the format of messages sent between the terminal and cash register is based on the C-TAP[1] message format (see [2]).

2.1.1 Tag Encoding

The tag is encoded with a variable number of bytes. Bits 1…5 (right most bit is bit 1) of the first byte determines if it is a single-byte or multi-byte tag. When bits 1…5 are all one, it is a multi-byte tag thus tags starting with 1F, 3F, 5F, 7F, 9F, BF, DF and FF are multi-byte tags.

Of the next tag bytes bit 8 determines if there are more tag-bytes. As long as bit 8 is set to one another tag byte will follow. Of the last tag byte bit 8 will be zero. Some example tags are shown in Table 1.

Tag (hex)	Description
D5	Bits 1…5 are not all set (single byte tag).
DF30	Bits 1…5 of the first byte are all set (multi-byte tag). Bit 8 of the second byte is not set so this is the last tag byte.
DF8101	Bits 1…5 of the first byte are all set (multi-byte tag). Bit 8 of the second byte is set thus another tag byte follows. Bit 8 of the third byte is not set so this is the last tag byte.

Table 1: Tag encoding examples

2.1.2 Length Encoding

The length is also encoded using a variable number of bytes. When the length is less than 128 bytes, the short form can be used and the length is encoded using one byte where bit 8 set to zero and bits 1…7 indicate the length (0…127).

The long form must be used for lengths of 128 bytes or longer but can also be used for shorter lengths. In the long form bit 8 of the first byte is set and bits 1…7 of the first byte specify how many length bytes follow. The following bytes encode the length where the first length byte is the most significant byte (big-endian coding).

Examples of length encoding are shown in Table 2.

[1] C-TAP is the protocol used for communication between the terminal and the payment processors.

Length coded (hex)	Length	Description
23	35	Short form (bit 8 not set).
7F	127	Short form (bit 8 not set).
817F	127	Long form (bit 8 of first byte set and bit 1…7 indicate one following length byte).
8193	147	Long form (bit 8 of first byte set and bit 1…7 indicate one following length byte).
821234	4660	Long form (bit 8 of first byte set and bit 1…7 indicate two following length bytes).

Table 2: Length encoding examples

2.2 Primitive Data Elements

The data of a TLV can be encoded in various formats. Which format is used depends on the tag. Each tag has its own data format so the tags used and their data format should be known in advance by both the sending and receiving parties.

In this documentation the data format is indicated by a letter preceded by the length. If the length is variable it is specified by two numbers indicating the minimum and maximum length. The length is not the same as the number of bytes. For example a bit field with length 8 takes only one byte. The data types that are defined are shown in Table 3.

Data type	Code	Example	Description
Bit field	b	8b	Field composed of a number of Boolean values. Each Boolean is one bit thus 8b (8 Booleans) take one byte. In this documentation we use LSB1 bit numbering whereby the right most bit is bit 1 (see bit numbering [5]).
ASCII string	a	1..10a	Field composed of ‘n’ ASCII characters. In the example the string consists of 1 to 10 characters. If the number of characters is less than the length, it will be left aligned and padded on the right with zeros.
BCD number	i	8i	Field composed of ‘n’ binary coded decimal (BCD) digits. The decimal digit values are in the range 0…9 and are coded in one nibble (4 bits) and packed into bytes with two nibbles per byte. Thus 8i (8 BCD digits) take 4 bytes. The most significant digit is on the left. The decimal digits are right justified and padded on the left with a 0 if needed.
HEX number	x	4x	Field composed of ‘n’ hexadecimal digits. The values are in the range 0...F and are coded in one nibble (4 bits) and packed into bytes with two nibbles per byte. Thus 4x (4 hexadecimal digits) take 2 bytes. The most significant digit is on the left. The hexadecimal digits are right justified and padded on the left with a 0 if needed. Thus 4x is a 16-bit integer in big-endian format.

Table 3: Data types

Examples of data type encoding are shown in Table 4.

Format	Value	Coded value (hexadecimal representation)
8b	00111101b	3D
8a	HELLO	48454C4C4F000000
8i	123456	00123456
8x	11211316 (0xAB1234)	00AB1234

Table 4: Data type encoding examples

2.3 Constructed Data Elements

Constructed data elements are TLVs of which the data consists of other TLVs thus enabling structured data. Whether a TLV is a constructed or primitive element is determined by bit 6 of the first tag byte (see the BER encoding [1]). When bit 6 of the first tag byte is zero, the TLV is a primitive data element. When bit 6 is one then the TLV is a constructed data element. There are two different types of constructed data elements namely groups and tables.

2.3.1 Group Field

A group field (represented by “GRP”) is a simple constructed data element that contains a variable number of other TLVs. The group can contain both TLVs with primitive data elements and TLVs with constructed data elements. Each group has its own tag. A group TLV does not necessarily contain other TLVs, an empty group is also possible.

Table 5 shows the coding of a hypothetical group TLV with tag A6 that contains two primitive TLVs with tags 4A (one hex digit) and 4C (variable length ASCII string).

Tag	Format	Content	Length	TLV Coding (hexadecimal representation)
A6	GRP	…	10	A6 0A …
4A	2x	0x01	1	4A 01 01
4C	0..1024a	HELLO	5	4C 05 48454C4C4F

Table 5: Example of constructed data: Group

2.3.2 Table Field

A table field (represented by “TBL”) is used for repeated data and consists of table records (represented by “REC”). A table record consists of one or more other TLVs. Each table has its own tag and is thus a special kind of group. A table record always has the tag “EF” so its data is determined by the table where the record is part of. Table records of a table do not need to contain the same tags.

Table 6 shows the coding of a hypothetical table TLV with two table records. Each record contains two primitive TLVs with tags 4A (one hex digit) and 4C (variable length ASCII string).

Tag	Format	Content	Length	TLV Coding (hexadecimal representation)
A6	TBL	…	24	A6 18 …
EF	REC	…	10	EF 0A …
4A	2x	0x01	1	4A 01 01
4C	0..1024a	HELLO	5	4C 05 48454C4C4F
EF	REC	…	10	EF 0A …
4A	2x	0x02	1	4A 01 02
4C	0..1024a	WORLD	5	4C 05 574F524C44

Table 6: Example of constructed data: Table

3 Messages and Message Structure

3.1 Overview

The terminal and cash register communicate via messages. This chapter describes the general message structure. Chapter 4 describes the different messages in detail.

The messages are sent over a physical connection. The physical connection can be an Ethernet connection (TCP/IP), serial connection or USB. Which physical connection the terminal uses for communication is determined by the connection string stored in the terminal's parameters. The connection string can be set by the Terminal Management System (TMS). The connection string also determines the communication parameters such as host/port for TCP connections or baud rate, data/stop bits and parity for serial connections. As the terminal's USB port is seen as a regular COM port on the device it is attached to, the USB communication is the same as serial communication.

In case of Ethernet the terminal is a TCP client while the cash register is a TCP server. This is done for security reasons as running a TCP server on the terminal is a potential security hole.

When there is a connection the terminal starts sending *HelloRequest* messages until it receives a *HelloResponse* message from the cash register. In the *HelloRequest*/*HelloResponse* the communication parameters are negotiated such as keep-alive interval, encryption parameters and the modification detection algorithm.

When the connection is established, the terminal regularly sends *KeepAliveRequest* messages to see if the connection is still up and also to notify the cash register of the terminal state.

After the *HelloRequest*/*HelloResponse* messages, the cash register can send commands to the terminal such as a *TransactionRequest* message. A typical message flow is given in Figure 1. Note that the keep-alive messages are sent parallel to the command messages from the cash register to the terminal.

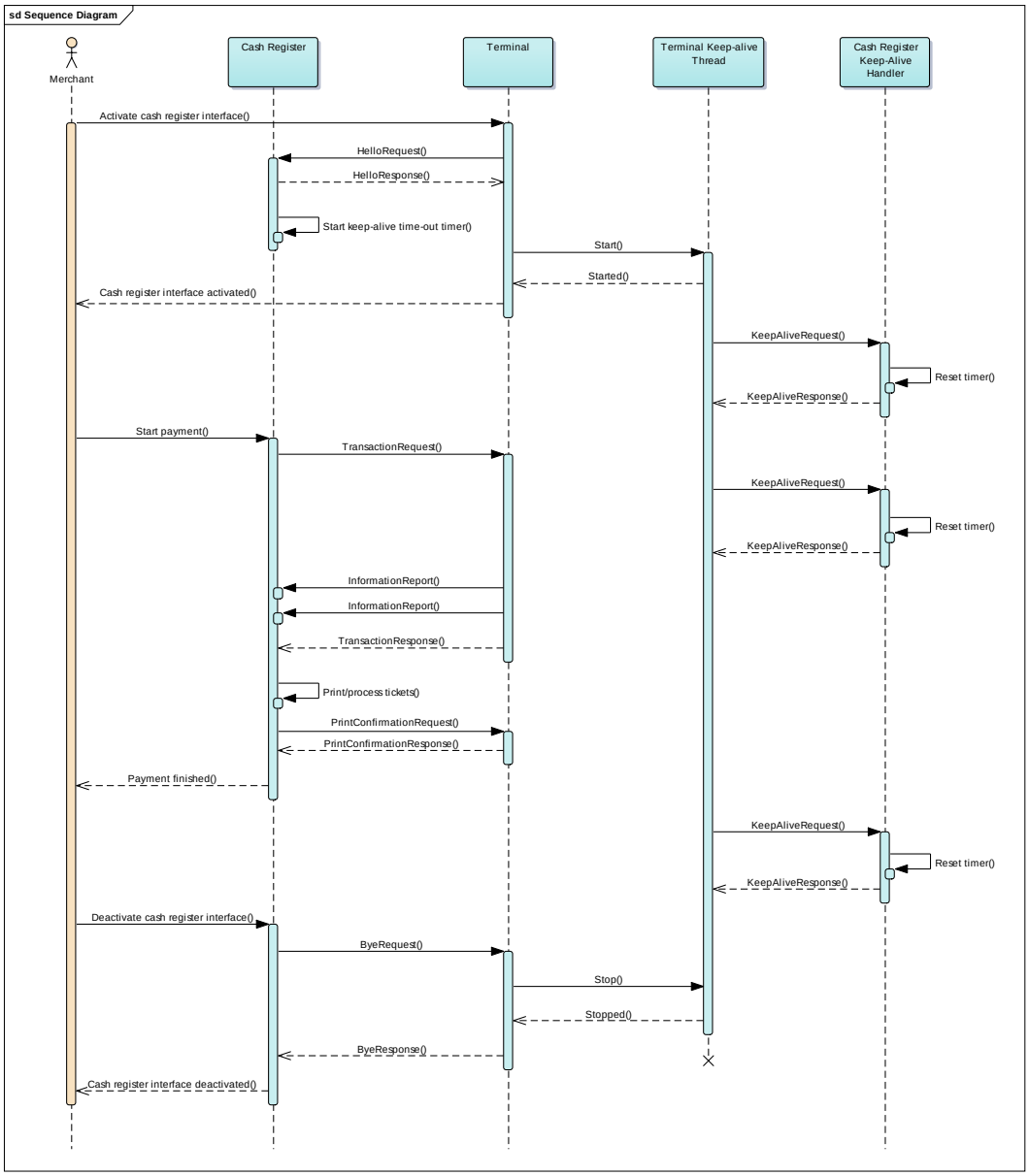


Figure 1: Basic message flow

3.2 Basic Message Structure

The packing of the TLV data into a message is the same as used with C-TAP (see [2]).
Any message consists of an F0 “root” group TLV that contains a header TLV, body TLV and footer TLV. In contrast to regular TLV groups, the order of header, body and footer is important. The header can contain encryption information which is important for parsing the body and footer.

The F0 “root” TLV is preceded by the length of the message data and the “magic” identifier “P4Y-ECR!” (50 34 59 2D 45 43 52 21). The message length field is a 32-bit integer in big-endian order (note that this is unlike the length of a TLV where the length field is encoded as a variable number of bytes). The message structure is depicted in Figure 2 and Table 7.

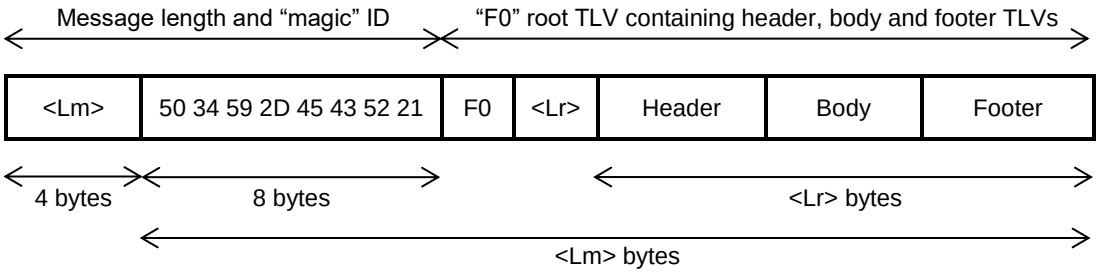


Figure 2: Message structure

			…	
Data Element	**Tag**	**Format**	**C-**	**R-**
The number of bytes that follow	-	8x	M	M
Magic ID “P4Y-ECR!”	-	8a	M	M
Message	F0	GRP	M	M
• Message Header	E1	GRP	M	M
• Message Body	E2	GRP	M	M
• Message Footer	E3	GRP	M	M

Table 7: Message layout

The last two columns in the table above indicate if a field is mandatory (M), optional (O), echoed (E) or conditional (C) for the request message (the C- column) or response message (the R- column):

- Mandatory: The field is required. Without it the message is not correct.
- Optional: The field is not required. It is only present in the message if it has a value.
- Echoed: The field is copied from the request message to the response message.
- Conditional: The field is only present if relevant. For example if encryption is chosen, then encryption parameters are required.

Note: When using a serial connection it is possible that the cash register receives bogus bytes when the terminal is switched on. This has been seen on the terminal’s COM0 port. The terminal’s COM2 and USB port do not seem to have this.
Because of the possibility of bogus bytes, it is recommended NOT to assume the first byte received is the start of a message. Instead read the length field and the magic ID field and check if the magic ID is “P4Y-ECR!”. If not there might be bogus bytes thus continue reading extra bytes until the magic ID is completely received discarding the bytes received before the length field and magic ID field.

3.3 Message Header

The message header group is mandatory for every message. It contains the protocol ID (protocol version), message ID, terminal and cash register (ECR) IDs and encryption parameters of the message. The structure of the header TLV is given in Table 8.

			…	
Data Element	**Tag**	**Format**	**C-**	**R-**
Message Header	E1	GRP	M	M
• Protocol ID	10	8i	M	M
• Message Type	11	8i	M	M
• Terminal ID	12	1..16a		
• ECR ID	13	1..16a		
• Sequence Number	14	4x	M	M
• Encryption Parameters	A0	GRP	C	C

Table 8: Message header structure

3.3.1 Protocol ID

The protocol ID is the version number of the cash register protocol. Currently there is only version 1. Future versions might add additional functionality. The version number should be checked before using that additional functionality. The cash register should return the version number of the "TLV Cash Register Interface Specification" it supports.

3.3.2 Message Type

The message type identifies the kind of message and thus affects the contents of the body. The currently defined messages are shown in Appendix B.1 (Table 46). Note that not all messages are implemented in the terminal yet.

Messages with an ID not recognised should be ignored. Also TLVs in a message that are not recognised should be ignored.

3.3.3 Terminal ID

This is the ID of the terminal which the terminal sends in every message. The cash register must echo this field. It is used to verify if the message is indeed intended for the current terminal (when sent from cash register to terminal). Or when sent from terminal to the cash register the cash register can verify that the sender is the same terminal as the one that established the connection in the *HelloRequest* message.

3.3.4 ECR ID

This is the ID of the cash register. The cash register must send this in every message. The terminal will echo this field. In the *HelloRequest* message this field is not given as the terminal does not yet know the ID of the cash register.

3.3.5 Sequence Number

Both the cash register and terminal must send a sequence number in their messages increasing every message. It can be used to detect if a message was missed. When the received sequence number is not as expected (the received sequence number is not one more than the sequence number in the previous received message) the connection should be disconnected.

The cash register and terminal have both their own sequence number. The sequence number is per connection thus for each new connection the counter starts at one. Note that the first message received does not need to have sequence number one because the terminal can send

HelloRequest messages over a serial connection when no cash register is connected yet. The *HelloResponse* must have sequence number 1.

Note that since the sequence number is a 16 bit integer, after sequence number 65535 the number wraps back to 0. When checking the current received number with the last received, this should be taken into account.

3.3.6 Encryption Parameters Group

The encryption parameters group is defined for future use. Future versions may add this group to pass the used encryption parameters. When implemented, encryption will be applied on the body TLV data (excluding the tag and length). Currently the message bodies are not encrypted.

Additionally this group may indicate the modification detection algorithm used (see section 3.5.1). When the modification detection algorithm is not given in the header, the CRC-32 algorithm must be used since that is the default modification detection algorithm.

3.4 Message Body

The message body group is mandatory for every message. It contains the primary data of the message. What data it contains depends on the message type (see section 3.3.2). The body content is described for each message in chapter 4.

3.5 Message Footer

The message footer group is mandatory for every message. It contains the modification detection code. The structure of the footer TLV is given in Table 9.

			…	
Data Element	**Tag**	**Format**	**C-**	**R-**
Message Footer	E3	GRP	M	M
• MDC	1C	8..128x	M	M

Table 9: Message footer structure

3.5.1 MDC

The modification detection code (MDC) is a checksum over the message body data. Which algorithm is used will in future versions be negotiated in the *HelloRequest*/*HelloResponse* messages using the “Encryption Parameters” tag (see section 4.1.1). In the first message and when no modification detection algorithm is negotiated in the *HelloRequest*/*HelloResponse* messages the default algorithm will be used. The default algorithm is CRC-32 which is described in Appendix C.

The MDC is calculated over the TLV message body data (excluding tag and length), before encryption. The size of the MDC depends on the algorithm used. For CRC-32 it is 32 bits (4 bytes, 8x). For SHA-512 it would be 512 bits (64 bytes, 128x).

4 Message Definitions

This chapter describes the different messages in detail. The generic parts of a message, the header and footer, are described in section 3.3 and 3.5 respectively. In this chapter only the body parts of the different messages are described.

Some examples of encoded message data is given in appendix E.

4.1 Hello Request/Response

The *HelloRequest*/*HelloResponse* messages are used to initiate the connection and to exchange connection parameters.

After a connection is established, the terminal starts sending *HelloRequest* messages until it receives a *HelloResponse* message from the cash register. With TCP/IP the *HelloRequest* message is usually only sent once since the connection is only established when both parties have opened the connection.
With a serial connection the terminal cannot see if the cash register also has opened the connection so it keeps sending *HelloRequest* messages until an answer is received.

Table 10 describes the structure of the *HelloRequest* and *HelloResponse* messages.

			Hello	
Data Element	**Tag**	**Format**	**C-**	**R-**
Message Header	E1	GRP	M	M
• Protocol ID	10	8i	M	M
• Message Type	11	8i	M	M
• Terminal ID	12	1..16a	M	E
• ECR ID	13	1..16a		M
• Sequence Number	14	4x	M	M
• Encryption Parameters	A0	GRP	O	O
Message Body	E2	GRP	M	M
• Encryption Parameters	A0	GRP	O	O
• Communication Parameters	A1	GRP	O	O
• Connection Parameters	A4	GRP	M	M
○ Keep-alive interval	49	4i	M	M
• Supported Languages	A2	TBL	M	
○ Table Record	EF	REC 1..n	M	
▪ Language (supported)	18	2a	M	
• Language (to be used)	18	2a		O
• Terminal Logical ID	81	1..16a	O	
• Terminal Serial Number	82	1..32a	O	
• Result Code	16	4x		M
Message Footer	E3	GRP	M	M
• MDC	1C	8..128x	M	M

Table 10: Hello request and response message structure

4.1.1 Encryption Parameters Group

The encryption parameters group is defined for future use. Future versions may add this group to the body to indicate which encryption methods are supported by the terminal (*HelloRequest*) and

which one to use (*HelloResponse*). When implemented, encryption will be applied on the body TLV data (excluding the tag and length). Currently the messages are not encrypted.

Additionally this group may indicate the modification detection supported by the terminal (*HelloRequest*) and which one to use (*HelloResponse*) (see section 3.5.1). When the MDC algorithm is not given, the CRC-32 algorithm must be used by default. Currently all messages use the CRC-32 algorithm for the MDC.

4.1.2 Communication Parameters Group

The communication parameters group is defined for future use. Future versions may add this group to indicate which communication methods are supported by the terminal (*HelloRequest*). In the *HelloResponse* message other communication parameters may be given after which the terminal disconnects and reconnects with the new parameters. This can for example be used to redirect the connection to another cash register.

4.1.3 Connection Parameters Group

The connection parameters group is used to exchange connection settings like the keep-alive interval.

4.1.3.1 Keep-alive Interval

This is the keep-alive interval in seconds.

After the *HelloRequest*/*HelloResponse* messages the terminal starts sending *KeepAliveRequest* messages (see section 0). In the *HelloRequest* message the terminal states the interval in which it wants to send keep-alive messages although there may be extra keep-alive messages when the terminal state changes (message with keep-alive reason “status” instead of “timer” (see section 4.2.1)).
The cash register usually echoes the value given by the terminal but it can return another value that the terminal should use thus overriding the value provided by the terminal.

A keep-alive interval of 0 means that no keep-alive messages are sent at regular intervals but keep-alive messages can still be sent when the terminal state changes (keep-alive reason “status” instead if “timer”).

4.1.4 Supported Languages Group

This is a table with the languages the terminal supports for informational messages to the cash register (merchant language). This language is different from the language that is used for messages shown on the terminal’s display to the card holder[1].

4.1.4.1 Language (supported)

The language field of a table is one of the merchant languages supported by the terminal as a two letter ISO 639-1 language code (see [3]), for example “nl” for Dutch or “en” for English.

[1] The language for card holder messages is a terminal setting but it might change after the card holder presents its card. The card might provide another preferred card holder language.

4.1.5 Language (to be used)

This is the language selected by the cash register as a two letter ISO 639-1 language code (see [3]), for example “nl” for Dutch or “en” for English. It must be one of the languages provided in the *HelloRequest* message (seen section 4.1.4). When no language is provided in the *HelloResponse* message the default terminal merchant language[1] will be used.

4.1.6 Terminal Logical ID

This is the logical ID of the terminal.

4.1.7 Terminal Serial Number

This is the serial number of the terminal (also called physical ID).

4.1.8 Result Code

The result code indicates the result of the request (success or the reason for failure). See Appendix B.2 (Table 47) for the possible result codes.

[1] The default terminal merchant language is a terminal setting.

4.2 Keep-alive Request and Keep-alive Response

Keep-alive messages are used to report the state of the terminal and to see if the connection is still up. The terminal is sending *KeepAliveRequest* messages while the cash register must respond by sending a *KeepAliveResponse* message.

The keep-alive interval is negotiated in the *HelloRequest*/*HelloResponse* messages (see section 4.1.3.1). When the cash register does not receive a keep-alive message in twice the negotiated interval time[1], it should disconnect and try to reconnect.
The terminal will disconnect if it does not receive a keep-alive response within the keep-alive interval.

Table 11 describes the structure of the *KeepAliveRequest* and *KeepAliveResponse* messages.

			Keep-alive	
Data Element	**Tag**	**Format**	**C-**	**R-**
Message Header	E1	GRP	M	M
• Protocol ID	10	8i	M	M
• Message Type	11	8i	M	M
• Terminal ID	12	1..16a	M	E
• ECR ID	13	1..16a	E	M
• Sequence Number	14	4x	M	M
• Encryption Parameters	A0	GRP	C	C
Message Body	E2	GRP	M	M
• Keep-alive Reason	15	2x	M	E
• Terminal State	19	16b	M	
• Result Code	16	4x	O	M
Message Footer	E3	GRP	M	M
• MDC	1C	8..128x	M	M

Table 11: Keep-alive request and response message structure

4.2.1 Keep-alive Reason

This is the reason of the keep-alive message (*KeepAliveRequest*). The keep-alive message can be a regular scheduled keep-alive or an extra keep-alive message when there was a status change (see Table 12).
The *KeepAliveResponse* message must echo the reason received in the keep-alive request.

Name	ID	Description
Timer	1	Keep-alive was sent because the keep-alive timer triggered.
Status	2	Keep-alive message was sent because the terminal status changed.

Table 12: Keep-alive reasons

[1] Because of processing and transfer delays the interval in which keep-alive messages are sent by the terminal is not accurate. Therefore when deciding if a keep-alive message is not received in time, the cash register should use a time longer than the keep-alive interval time. It is recommended to use twice the interval time because that provides a margin for the delays and is before the next keep-alive is sent.

4.2.2 Terminal State

This indicates the state of the terminal. It is a bit field indicating which operations are possible, if the terminal's printer is available and if the battery is low. The meaning of the bits is described in Table 13. If the terminal state indicates that an operation is not possible and that operation is requested, a "rejected" is returned as the result code.

Bit	Meaning
1	When set a "transaction" request possible.
2	When set a "last transaction" request possible.
3	When set a "last ticket" request possible.
4	When set a "totals" request possible.
5	When set a "bye" request possible.
6	When set the terminal battery is low (for mobile terminals).
7	When set a "cancel" request is possible.
8	When set the terminal printer is available.
9-16	- Reserved for future use – (currently always 0).

Table 13: Terminal state bit meanings

4.2.3 Result Code

The result code in the *KeepAliveRequest* message can give the reason why not all operations are available when the keep-alive request reason was "*Status*". The result code is always "*OK*" when the keep-alive request reason was "Timer".

See Appendix B.2 (Table 47) for the possible result codes.

4.3 Bye Request/Response

The *ByeRequest* message is sent when one party wants to break the connection. It can be sent by both the terminal and cash register.

The party that sends the *ByeRequest* may only break the connection after receiving the *ByeResponse* message. Just sending the *ByeRequest* message is not sufficient.

The party that receives the *ByeRequest* message must reply by sending the *ByeResponse* message after which it can break the connection.

Table 14 describes the structure of the *ByeRequest* and *ByeResponse* messages.

			Bye	
Data Element	**Tag**	**Format**	**C-**	**R-**
Message Header	E1	GRP	M	M
• Protocol ID	10	8i	M	M
• Message Type	11	8i	M	M
• Terminal ID	12	1..16a	M/E	E/M
• ECR ID	13	1..16a	E/M	M/E
• Sequence Number	14	4x	M	M
• Encryption Parameters	A0	GRP	C	C
Message Body	E2	GRP	M	M
• Bye Reason	17	2x	M	E
• Result Code	16	4x		M
Message Footer	E3	GRP	M	M
• MDC	1C	8..128x	M	M

Table 14: Bye request and response message structure

4.3.1 Bye Reason

The bye reason indicates the reason for the disconnection when sent from the terminal to the cash register. When sent from the cash register to the terminal it indicates what the terminal must do after disconnecting. The possible values are described in Table 15.

Name	ID	Description
Deactivate	1	Deactivate the cash register interface (from ECR to terminal and from terminal to ECR).
Shutdown	2	Terminal must do a shutdown (from ECR to terminal). Reconnect after reboot.
Reboot	3	Terminal must do a reboot (from ECR to terminal) or is performing a reboot (from terminal to ECR e.g. when a software update is started). Reconnect after reboot.
Reconnect	4	Terminal must reconnect (from ECR to terminal). Can be used to use new settings for a connection.
Battery Empty	5	Terminal is performing a shutdown because of a low battery (from terminal to ECR).

Table 15: Bye reasons

4.3.2 Result Code

This is the result of the *ByeRequest* message. It can be “Rejected” when the terminal is busy with another operation. When the terminal initiated the *ByeRequest*, the cash register must always return “*OK*” as the terminal will always disconnect[1].

See Appendix B.2 (Table 47) for the possible result codes.

[1] The terminal only initiates a disconnect when the connection is explicitly disabled by the merchant where it forcefully disconnects, or when the battery is empty in which case it must disconnect because the terminal shuts down. Normally the cash register takes the initiative to disconnect.

4.4 Information Report

During a transaction the terminal may[1] provide progress report messages like “waiting for card”, “waiting for PIN”, etc. Also important messages for the merchant can be sent like a message saying that the identity of the card holder should be checked. This message is only sent from terminal to cash register. There is no response message.

The *InformationReport* is also “misused” to return “message parsing” errors to the ECR. This is useful for developers when implementing the TLV interface on an ECR. When the message is invalid (e.g. required tag missing, tag length not as expected, etc.), an *InformationReport* is sent with a merchant text describing the problem. A correctly implemented ECR should not trigger these “error reports”. When an invalid ECR message is received, the message is ignored.

Table 16 describes the structure of the *InformationReport* message.

			Information	
Data Element	**Tag**	**Format**	**C-**	**R-**
Message Header	E1	GRP	M	
• Protocol ID	10	8i	M	
• Message Type	11	8i	M	
• Terminal ID	12	1..16a	M	
• ECR ID	13	1..16a	E	
• Sequence Number	14	4x	M	
• Encryption Parameters	A0	GRP	C	
Message Body	E2	GRP	M	
• Transaction ID	1A	16x	E	
• Terminal State	19	16b	M	
• Transaction Stage	0E	2x	M	
• Stage Message	0F	0..41a	O	
• Merchant Text	1D	0..41a	O	
Message Footer	E3	GRP	M	
• MDC	1C	8..128x	M	

Table 16: Information report message structure

4.4.1 Transaction ID

This is the ID of the transaction in progress. The ID was given in the *TransactionRequest* message (see section 4.5.1) and echoed here.

4.4.2 Terminal State

Same as described in section 4.2.2.

4.4.3 Transaction Stage

This is the current transaction stage (waiting for card, waiting for PIN, etc.). See Appendix B.3 (Table 48) for the possible values. Normally it is not needed to use this value. Instead use the “stage message” text (see section 4.4.4) since that provides a transaction stage message in the

[1] Currently the Ingenico terminal software does not yet send progress report messages.

merchant language. But in case the terminal does not support the required language, the transaction stage could be used to create your own message.

4.4.4 Stage Message

This is a message describing the current transaction stage. It can be used as status text on a cash register.

4.4.5 Merchant Text

A merchant text can be sent from the payment processor to the terminal and can contain important information for the merchant and occurs only in special situations. For example a message saying that the merchant should check the ID of the card holder (“Check ID”). Therefore the merchant text must be displayed to the merchant and the merchant must acknowledge the message. An unattended terminal (e.g. in a vending machine) does not produce merchant texts.

4.5 Transaction Request/Response

The *TransactionRequest* message is sent when the cash register wants the terminal to perform a transaction. While the terminal is performing a transaction it can send *InformationReport* messages back to the cash register. When the transaction is finished the terminal sends a *TransactionResponse* message to the cash register with the transaction outcome and the transaction receipt(s) for integration with the cash register receipt.

Table 17 describes the structure of the *TransactionRequest* and *TransactionResponse* messages.

			Transaction	
Data Element	**Tag**	**Format**	**C-**	**R-**
Message Header	E1	GRP	M	M
• Protocol ID	10	8i	M	M
• Message Type	11	8i	M	M
• Terminal ID	12	1..16a	E	M
• ECR ID	13	1..16a	M	E
• Sequence Number	14	4x	M	M
• Encryption Parameters	A0	GRP	C	C
Message Body	E2	GRP	M	M
• Transaction ID	1A	16x	M	E
• Transaction Data	A3	GRP	M	
○ Amount	50	8i	M	
○ Currency Code	51	4i	O	
○ Currency Exponent	52	2i	O	
○ Cardholder Language	DF1A	2a	O	
○ Progress Report Language	53	2a	O	
○ Transaction Type	54	2x	O	
○ Merchant Transaction Reference	55	0..16a	O	
○ Intermediate Result Mode	58	2x	O	
○ Transaction Mode (only for testing)	59	2x	O	
○ Authorisation Code	1F70	6a	O	
○ Token	1F71	1..19a	O	
• Print Data	A5	GRP	M	
○ Print Origin	4E	2x	M	
○ Paper Width	4F	2i	O	
• Terminal State	19	16b		M
• Transaction Result	56	2x		M
• Card Brand Identifier	DF5F	4i		O
• Ticket Data	A6	TBL		O
○ Table Record	EF	REC 0..3		M
▪ Ticket Type	4A	2x		M
▪ Ticket Header	4B	0..1024a		O
▪ Ticket Body	4C	0..2048a		M
▪ Ticket Footer	4D	0..1024a		O
• Result Code	16	4x		M
Message Footer	E3	GRP	M	M
• MDC	1C	8..128x	M	M

Table 17: Transaction request and response message structure

4.5.1 Transaction ID

The transaction ID is a cash register generated identifier for the transaction. It will be echoed by the terminal in the *TransactionResponse* message and in the *InformationReport* message (see section 4.4.1)

4.5.2 Transaction Data Group

The transaction data group contains the data required to perform a transaction.

4.5.2.1 Amount

This is the amount of the transaction. When the currency is Euro, the amount unit is in cents.

4.5.2.2 Currency Code

The currency code indicates the currency of the transaction. This is an ISO 4217 3-digit numeric currency code e.g. 978 for Euro (see [4]). When not provided the terminal's default currency is used. The default currency depends in which country the terminal is operating and for the Euro zone the default currency will be Euro.

4.5.2.3 Currency Exponent

The currency exponent indicates the 10^{exp} multiplication factor for the amount. In case of Euro it is 2 because the amount unit is in cents (euro amount * 10^2 = amount in cents for transaction).
In other words it indicates the position of the decimal separator from the right in the transaction amount value.

The currency exponent is part of the ISO 4217 currency code definitions (see [4]) where it indicates the relation between the currency's major and minor unit (e.g. Euro and Euro cents). As such it is also used in C-TAP (see [2]).

When not provided, the default for the currency is used as specified by ISO 4217.

4.5.2.4 Card Holder Language

The card holder language is the initial language for card holder messages. When given it will override the default card holder language terminal setting. It is a two letter ISO 639-1 code (see [3]), for example 'nl', 'en', etc. This language should be one of the languages that were given by the terminal in the *HelloRequest* and will only be used for the current transaction.

Note that when the card holder presents his card, the card holder language could be changed to the preferred language stored on the card.

4.5.2.5 Progress Report Language

The progress report language is the language to use for progress and merchant messages in case this language should be different from the one set in the *HelloResponse* (see section 4.1.5). It is a two letter ISO 639-1 code (see [3]), for example 'nl', 'en', etc. This language should be one of the languages that were given by the terminal in the *HelloRequest* and will only be used for the current transaction.

4.5.2.6 Transaction Type

This is the type of transaction (payment, refund, etc.). The possible values are given in Table 18 and are based on C-TAP.220 Table 49 (see [2]). Which transaction types are supported depends on your contract with the payment provider and what is enabled by the terminal provider.
Unattended terminals only support payment and deferred payment.
When not provided, the terminal will perform a regular payment.

Name	ID	Description
Payment	1	Payment (sale) transaction.
CashAdvance	2	Cash advance transaction.
PreAuthorization	3	Pre-authorisation (reservation) of funds.
PaymentCompletion	4	Payment completion (after pre-authorisation) (sale after reservation of funds).
CardValidityCheck	5	Card validity check.
Refund	6	Refund transaction.
DeferredPayment	7	Deferred payment (sale) transaction. With a deferred payment, the initial amount is ignored (should be zero) and the intermediate result request returns the authorised amount (the maximum amount that can be spent). In the intermediate result response the ECR sends the final amount for the transaction. Deferred sale can be used in vending machines where the amount is only known after delivery e.g. petrol stations.
~Reserved~	8	Reserved for future use.
Cancellation	9	Cancellation of for example reservation.
CancellationPayment	10	Cancellation of payment transaction.
CancellationCashAdvance	11	Cancellation of cash advance transaction.
CancellationPreAuthorization	12	Cancellation of pre-authorisation transaction.
CancellationPaymentCompletion	13	Cancellation of payment completion transaction.
~Reserved~	14	Reserved for future use.
~Reserved~	15	Reserved for future use.
UpdatePreAuthorization	16	Update Pre-authorization (reservation) of funds.
OriginalCredit	17	Original credit

Table 18: Transaction types

4.5.2.7 Merchant Transaction Reference

This is an optional arbitrary reference for the transaction that could be entered by the merchant or generated by the cash register. For example a code derived from the data and a sequence number. The merchant reference will be sent to the payment host and will be visible on the card holder bank statement.

4.5.2.8 Intermediate Result Mode

This is an optional indicator to inform the terminal to report the intermediate result to the cash register before the transaction is finalised. This is used in vending machines where the payment must be cancelled when the product cannot be delivered after the payment was authorised by replying with the “vending result”.
It can also be used when knowing if the transaction is authorised is time critical (high volume vending) so the product can already be prepared without waiting for the transaction completion. After the transaction is finalised with the payment host, the complete transaction result will be sent containing the ticket(s).

Table 19 shows the possible intermediate result modes.

See section 0 for more information on handling the intermediate result request.

Name	ID	Description
None	0	Do not use intermediate result. This is the same as not providing the intermediate result tag. Only the final transaction results are sent in a *TransactionResponse* message.
IntermediateResult	1	Use intermediate result. The terminal will send an *IntermediateResultRequest* to the ECR when it receives the authorisation response from the payment host. The ECR must respond to the *IntermediateResultRequest* with the vending result (product delivered or not). When the product is delivered, the transaction will be finalised by sending a completion advise to the payment host and money will be transferred. In case the product has not been delivered, a negative advice will be sent to the payment host and no money will be transferred. This mused is thus to be used when product delivery can fail.
IntermediateResultNoResponse	2	Use intermediate result without the need for response. The terminal will send an *IntermediateResultRequest* to the ECR when it receives the authorisation response from the payment host but in this case the ECR must NOT respond to the *IntermediateResultRequest*. The transaction is automatically confirmed with the payment host. This mode can be used if it is needed to know if the payment is authorised as soon as possible and there is no need to cancel the transaction if the vending failed.

Table 19: Intermediate result modes

4.5.2.9 Transaction Mode

This field is only for testing. Do not provide this field as the transaction mode is normally determined by the terminal.

4.5.2.10 Authorisation Code

The authorisation code is a 6 digit alphanumeric code which is needed if the transaction refers to a previous transaction. It is printed on the transaction ticket of a successful transaction and needs to be entered if the transaction is cancelled or a pre-authorisation is updated or confirmed.

Transaction types that need an authorisation code are: *UpdatePreAuthorisation*, *PaymentCompletion* and all types of cancellation. Note that *Refund* does not need an authorisation code because that is not linked to a previous transaction. A refund transaction could also be done if the original transaction was a cash payment.

4.5.2.11 Token

When a transaction is done that refers to a previous transaction (cancellation or a pre-authorisation is updated or confirmed), the original card needs to presented again. However the original card might not always be present. In those cases the payment host[1] might have provided a token that is printed on the ticket of the original transaction. The token refers to the original card data that is stored at the payment host. Thus instead of presenting the original card, the token can be provided.

The token is a numeric string up to 19 characters and must fulfil the “Luhn mod 10 plus 1” test. This is a variant of the Luhn algorithm (see [6]) where the check digit is incremented by one (thus when checking the code, the check digit should first be decremented by one first).

It is recommended that the ECR performs the Luhn check on the token before sending it to the terminal to avoid failed transactions. When the token is invalid the terminal will return a *ParameterInvalid* error code. Note that the check must be performed on the binary value of the number, not the ASCII value. See Appendix D for implementation details of the Luhn check.

4.5.3 Print Data Group

The print data group contains the parameters for generating and printing the various receipts.

4.5.3.1 Print Origin

The print origin indicates the place where the transaction ticket(s) should be printed. The possible values are given in Table 20.

Name	ID	Description
Terminal	1	Print the ticket on the terminal (only if the terminal has a printer).
CashRegister	2	Send the ticket to the cash register.
TerminalAndCashRegister	3	Print the ticket on the terminal (only if the terminal has a printer) and send the ticket to the cash register.

Table 20: Print origin codes

4.5.3.2 Paper Width

This is the width of the printer paper in the cash register in characters. It is only used for the tickets sent to the cash register. When the ticket is printed on the terminal the width of the terminal printer is used (usually 24 characters). The width must be between 24 and 98 characters. When the paper width is not given, the terminal printer width is used (by default 24 characters).

4.5.4 Terminal State

Same as described in section 4.2.2.

4.5.5 Transaction Result

This is the result of the transaction. The possible values are given in Table 21.

[1] Not all payment hosts support tokens.

Name	ID	Description
Approved	0	The transaction succeeded.
Error	1	The transaction failed. See the Result Code for reason.

Table 21: Transaction result codes (transaction response)

4.5.6 Card Brand Identifier

The card brand identifier is the type of card that is used for the payment. The possible values are given in table Table 22.

Identifier	Description
1001	Bancontact/MisterCash (Domestic)
1002	Visa Electron
1003	V-Pay
1009	Maestro
1011	WL Company cards – domestic
1012	WL Company cards – international
1013	Bancontact
1020	Cartes Bancaires
1030	Payconic
1040	GoodPay
2002	Visa
2003	MasterCard
2004	AMEX
2005	Diners/Discover
2007	JCB
2008	UnionPay
2009	Monizze
2012	CMFC
2014	Basic Card
3001	BuyWay
3002	PASS
3003	Sodexo
3004	Edenred
3013	CCV Card
3014	Travelcard
3020	Company Cards
3021	Worldline WL
3100	Equens WL
3200	Yourgift
3300	Giftcard
3400	Kadoz
3401	RES

Table 22: Card Brand Identifiers

4.5.7 Ticket Data Table

The ticket data is a table containing zero or more tickets (card holder, merchant and journal tickets). Note that even when the transaction failed, a journal ticket can still be returned. For example when the transaction is rejected by the host because the card is blocked or if there is insufficient balance).

4.5.7.1 Ticket Type

The ticket type indicates what type of ticket the “ticket data” contains. The possible ticket types are given in Table 23.

Name	ID	Description
Merchant	1	Optional transaction ticket for the merchant. Usually only generated in case the transaction requires a card holder signature.
CardHolder	2	Card holder transaction ticket which can be integrated with the cash register receipt. It is usually generated for magstripe and contact chip transactions. In some applications a card holder ticket is not generated for example for low-value (contactless) transactions at a vending machine.
Journal	3	Optional journal ticket for administration. When the terminal is configured for electronic journals, no journal ticket will be sent to the cash register but will be stored in an electronic journal system.
Totals	4	Ticket sent in case of a totals request (see section 0).

Table 23: Ticket types.

4.5.7.2 Ticket Header

This is the header text of the ticket. Currently empty but might contain data in future versions.

4.5.7.3 Ticket Body

This is the body text of the ticket. It contains the transaction details.

4.5.7.4 Ticket Footer

This is the footer text of the ticket. Currently empty but might be might contain data in future versions.

4.5.8 Result Code

The result code indicates the result of the request (success or the reason for failure). It can contain extra information why the transaction was not approved.

See Appendix B.2 (Table 47) for the possible result codes.

4.6 Intermediate Result Request/Response

Intermediate result request is send by the terminal to the ECR during a transaction in case of a “Deferred Payment” or a regular payment that was started with the “Intermediate Result Mode” field set to “IntermediateResult” or “IntermediateResultNoResponse”. It is send after the payment host has approved the transaction, but the transaction is not finished yet. The completion advice is still to be sent and the transaction can still be declined. The *IntermediateResultResponse* is send by the ECR to the terminal with in the transaction result code “Approved” or “Error”.

Table 24 describes the structure of the *IntermediateResultRequest* and *IntermediateResultResponse* messages.

			Intermediate Result	
Data Element	Tag	Format	C-	R-
Message Header	E1	GRP	M	M
• Protocol ID	10	8i	M	M
• Message Type	11	8i	M	M
• Terminal ID	12	1..16a	E	M
• ECR ID	13	1..16a	M	E
• Sequence Number	14	4x	M	M
• Encryption Parameters	A0	GRP	C	C
Message Body	E2	GRP	M	M
• Transaction Result	56	2x	M	M
• Amount	50	8i	M	C
• Intermediate Result Mode	58	2x	O	
Message Footer	E3	GRP	M	M
• MDC	1C	8..128x	M	M

Table 24: Intermediate result request and response message structure

4.6.1 Transaction Result

In the request (send by the terminal) it will always be “Approved” (otherwise the request would not have been sent).
In the response the ECR will decide whether the transaction is “Approved” or not (“Error”) (See Table 21). For example, depending on if the product could be delivered an “Approved” or “Error” is sent to the terminal.

4.6.2 Amount

In case of a regular sale with “intermediate result” enabled, the amount is 0 in the request and will be ignored by the terminal in the response (no need to send).

In case of a “deferred sale”, the terminal will send the amount that is approved by the host (the maximum amount that may be spent). The ECR shall send the actual amount to be payed (less or equal to the amount sent by the terminal).

4.6.3 Intermediate Result Mode

In case the transaction was started with the “intermediate result mode” set, it will echo the intermediate result mode. This can be used to determine if an *IntermediateResultResponse* message should be sent back to the terminal (mode = “IntermediateResult”) or not (mode = “IntermediateResultNoResponse”).

When the transaction type was a “deferred payment”, the “intermediate result mode” is not returned.

4.7 Cancel Request/Response

After a transaction request is sent, it can be cancelled by the cash register if the terminal is still waiting for a card. To cancel a transaction, send a *CancelRequest* after which the terminal replies with a *CancelResponse*.

Table 25 describes the structure of the *CancelRequest* and *CancelResponse* messages.

			Cancel	
Data Element	Tag	Format	C-	R-
Message Header	E1	GRP	M	M
• Protocol ID	10	8i	M	M
• Message Type	11	8i	M	M
• Terminal ID	12	1..16a	E	M
• ECR ID	13	1..16a	M	E
• Sequence Number	14	4x	M	M
• Encryption Parameters	A0	GRP	C	C
Message Body	E2	GRP	M	M
• Cancel reason	1E	2x	M	E
• Terminal State	19	16b		M
• Result Code	16	4x		M
Message Footer	E3	GRP	M	M
• MDC	1C	8..128x	M	M

Table 25: Cancel request and response message structure

4.7.1 Cancel Reason

This is the reason why the operation is being cancelled. The possible values are given in Table 46.

Name	ID	Description
Manual	0	The merchant manually requested the cancellation.
System	1	The cash register system requested the cancellation.

Table 26: Cancel reason codes

The terminal will echo the reason.

4.7.2 Terminal State

Same as described in section 4.2.2.

4.7.3 Result Code

The result code indicates if the cancel request was accepted (code "*ResultOK*"), rejected because cancellation is not allowed (code "*Rejected*") or failed because the cancellation request could not be performed for some reason (code "*CancelFailed*").
It does not indicate if the cancellation operation itself was successful. To see if the cancellation was successful, the result code of the operation should be checked for result codes

“*CtapCustomerAborted*” and “*CtapMerchantAborted*”. The distinction between merchant and card holder is not always made.

4.8 Last Transaction Ticket Request/Response

If for some reason printing of the ticket failed or a copy of the card holder ticket is needed, the card holder ticket of the last transaction can be requested from the terminal using a *LastTransactionTicketRequest* message.

Table 27 describes the structure of the *LastTransactionTicketRequest* and *LastTransactionTicketResponse* messages.

			Last Transaction Ticket	
Data Element	**Tag**	**Format**	**C-**	**R-**
Message Header	E1	GRP	M	M
• Protocol ID	10	8i	M	M
• Message Type	11	8i	M	M
• Terminal ID	12	1..16a	E	M
• ECR ID	13	1..16a	M	E
• Sequence Number	14	4x	M	M
• Encryption Parameters	A0	GRP	C	C
Message Body	E2	GRP	M	M
• Transaction ID	1A	16x	O	O
• Print Data	A5	GRP	M	
○ Print Origin	4E	2x	M	
○ Paper Width	4F	2i	O	
• Terminal State	19	16b		M
• Ticket Data	A6	TBL		O
○ Table Record	EF	REC 0..3		M
▪ Ticket Type	4A	2x		M
▪ Ticket Header	4B	0..1024a		O
▪ Ticket Body	4C	0..2048a		M
▪ Ticket Footer	4D	0..1024a		O
• Result Code	16	4x		M
Message Footer	E3	GRP	M	M
• MDC	1C	8..128x	M	M

Table 27: Last transaction ticket request and response message structure

4.8.1 Transaction ID

In the request, the ID of the transaction to show. It is used in the terminal to check if the last transaction in the terminal is the same as requested. When not provided the terminal does not check the ID. When given in the response, the transaction ID known to the terminal. If not returned the transaction ID was unknown.

4.8.2 Print Data Group

The printing parameters. See section 4.5.3 for a description.

4.8.3 Terminal State

Same as described in section 4.2.2.

4.8.4 Ticket Data Table

The returned ticket(s). See section 4.5.7 for a description.

4.8.5 Result Code

The result code indicates the result of the request (success or the reason for failure). When there was no last transaction or the given transaction ID was not the last transaction, “*NotAvailable*” is returned.

4.9 Last Transaction Status Request/Response

When there is uncertainty about the result of the last transaction, a *LastTransactionStatusRequest* can be sent to the terminal. The terminal will then send the last transaction details to the cash register and will also show the details to the card holder on the terminal display.

Table 28 describes the structure of the *LastTransactionStatusRequest* and *LastTransactionStatusResponse* messages.

			Last Transaction Status	
Data Element	**Tag**	**Format**	**C-**	**R-**
Message Header	E1	GRP	M	M
• Protocol ID	10	8i	M	M
• Message Type	11	8i	M	M
• Terminal ID	12	1..16a	E	M
• ECR ID	13	1..16a	M	E
• Sequence Number	14	4x	M	M
• Encryption Parameters	A0	GRP	C	C
Message Body	E2	GRP	M	M
• Transaction ID	1A	16x	O	O
• Terminal State	19	16b		M
• Result Code	16	4x		M
• Transaction Information	A8	GRP		O
○ Amount	50	8i		M
○ Currency Code	51	4i		M
○ Currency Exponent	52	2i		M
○ Transaction Result	56	2x		M
○ Transaction Date/Time	57	7i		M
Message Footer	E3	GRP	M	M
• MDC	1C	8..128x	M	M

Table 28: Last transaction status request and response message structure

4.9.1 Transaction ID

In the request, this is the ID of the transaction to show. It is used in the terminal to check if the last transaction in the terminal is the same as requested. When not provided the terminal does not check the ID. When given in the response, the transaction ID known to the terminal. If not returned the transaction ID was unknown.

4.9.2 Terminal State

Same as described in section 4.2.2.

4.9.3 Result Code

The result code indicates the result of the request (success or the reason for failure). When there was no last transaction or the given transaction ID was not the last transaction, "*NotAvailable*" is returned.
It does not indicate the reason of a transaction failure.

4.9.4 Transaction Information Group

This contains the information of the last transaction if it was available.

4.9.4.1 Amount

This is the amount of the last transaction. When the currency is Euro, the amount unit is in cents (currency exponent is 2).

4.9.4.2 Currency Code

The currency code indicates the currency of the last transaction. This is an ISO 4217 3-digit numeric currency code e.g. 978 for Euro (see [4]).

4.9.4.3 Currency Exponent

The currency exponent indicates the 10^{exp} multiplication factor for the amount. In case of Euro it is 2 because the amount unit is in cents (transaction amount in cents / 10^2 = euro amount of the transaction).

The currency exponent is part of the ISO 4217 currency code definitions (see [4]) where it indicates the relation between the currency's major and minor unit (e.g. Euro and Euro cents). As such it is also used in C-TAP (see [2]).

4.9.4.4 Transaction Result

This is the result of the transaction. The possible values are given in Table 21.

Name	ID	Description
Approved	0	The transaction succeeded.
Error	1	The transaction failed.
Stopped	2	The transaction was aborted by the card holder or merchant.
Error	3	There was an error during the transaction.
TechnicalProblem	4	There was a technical problem during the transaction.

Table 29: Transaction result codes (transaction information response)

4.9.4.5 Transaction Date/Time

The date/time of the transaction.

4.10 Totals Request/Response

Various totals can be requested from the terminal using the *TotalsRequest* message. The *TotalsResponse* will contain a totals ticket.

Table 30 describes the structure of the *TotalsRequest* and *TotalsResponse* messages.

			Totals	
Data Element	**Tag**	**Format**	**C-**	**R-**
Message Header	E1	GRP	M	M
• Protocol ID	10	8i	M	M
• Message Type	11	8i	M	M
• Terminal ID	12	1..16a	E	M
• ECR ID	13	1..16a	M	E
• Sequence Number	14	4x	M	M
• Encryption Parameters	A0	GRP	C	C
Message Body	E2	GRP	M	M
• Terminal State	19	16b		M
• Totals Type	5F54	2i	M	E
• Progress Report Language	53	2a	O	
• Print Data	A5	GRP	M	
○ Print Origin	4E	2x	M	
○ Paper Width	4F	2i	O	
• Ticket Data	A6	TBL		O
○ Table Record	EF	REC 0..3		M
▪ Ticket Type	4A	2x		M
▪ Ticket Header	4B	0..1024a		O
▪ Ticket Body	4C	0..2048a		M
▪ Ticket Footer	4D	0..1024a		O
• Result Code	16	4x		M
Message Footer	E3	GRP	M	M
• MDC	1C	8..128x	M	M

Table 30: Totals request and response message structure

4.10.1 Terminal State

Same as described in section 4.2.2.

4.10.2 Totals Type

The type of totals to retrieve. See Table 31 for the possible values.

ID	Description
1	X-totals (optionally returns Terminal Counter Group)
2	Z-totals (optionally returns Terminal Counter Group)
3	Current period totals (optionally returns C-TAP Counter Group)
4	Previous period totals (optionally returns C-TAP Counter Group)
5	Current and previous period totals (optionally returns C-TAP Counter Group)

Table 31: Totals type second nibble; totals type options

4.10.3 Print Data Group

The printing parameters. See section 4.5.3 for a description.

4.10.4 Ticket Data Table

The returned totals ticket. See section 4.5.7 for a description of the ticket data.

4.10.5 Result Code

The result code indicates the result of the request (success or the reason for failure).

4.11 Security Scheme List Request/Response

This is a request from the ECR to the terminal. The ECR asks C-TAP to give a list of all available security schemes. The use of this list is to choose one security scheme to do a download for.

Table 32 describes the structure of the *SecuritySchemeListRequest* and *SecuritySchemeListResponse* messages.

			Security Scheme List	
Data Element	**Tag**	**Format**	**C-**	**R-**
Message Header	E1	GRP	M	M
• Protocol ID	10	8i	M	M
• Message Type	11	8i	M	M
• Terminal ID	12	1..16a	E	M
• ECR ID	13	1..16a	M	E
• Sequence Number	14	4x	M	M
• Encryption Parameters	A0	GRP	C	C
Message Body	E2	GRP	M	M
• Security Scheme List	BF01	GRP		M
○ Table Record	EF	REC 0..x		O
▪ Security Scheme Identifier	DF8204	8x		M
Message Footer	E3	GRP	M	M
• MDC	1C	8..128x	M	M

Table 32: Security Scheme List request and response message structure

4.11.1 Security Scheme List

A table containing a list of all available security schemes

4.11.1.1 Security Scheme Identifier

The ID of the security scheme.

4.12 Security Scheme Download Request

This is a request from the ECR to the terminal. The ECR asks C-TAP to perform a download (key load) of the security scheme, identified by the Security Scheme Identifier. There is no response message but information reports can be sent.

Table 33 describes the structure of the *SecuritySchemeDownloadRequest* message.

			Security Scheme Download	
Data Element	**Tag**	**Format**	**C-**	**R-**
Message Header	E1	GRP	M	
• Protocol ID	10	8i	M	
• Message Type	11	8i	M	
• Terminal ID	12	1..16a	E	
• ECR ID	13	1..16a	M	
• Sequence Number	14	4x	M	
• Encryption Parameters	A0	GRP	C	
Message Body	E2	GRP	M	
• Security Scheme Identifier	DF8204	8x	M	
Message Footer	E3	GRP	M	
• MDC	1C	8..128x	M	

Table 33: Security Scheme Download request message.

4.12.1 Security Scheme Identifier

The ID of the security scheme to perform a download for.

4.13 Acquirer List Request/Response

This is a request from the ECR to the terminal. The ECR asks C-TAP to give a list of all available acquirers. The use of this list is to choose one acquirer to do a download for.

Table 34 describes the structure of the *AcquirerListRequest* and *AcquirerListResponse* messages.

			Acquirer List	
Data Element	**Tag**	**Format**	**C-**	**R-**
Message Header	E1	GRP	M	M
• Protocol ID	10	8i	M	M
• Message Type	11	8i	M	M
• Terminal ID	12	1..16a	E	M
• ECR ID	13	1..16a	M	E
• Sequence Number	14	4x	M	M
• Encryption Parameters	A0	GRP	C	C
Message Body	E2	GRP	M	M
• Acquirer List	E7	GRP		M
○ Table Record	EF	REC 0..x		O
▪ Acquirer Identifier	DF68	8i		M
▪ Acquirer Label Name	DF69	0..16a		M
Message Footer	E3	GRP	M	
• MDC	1C	8..128x	M	

Table 34: Acquirer list request and response message structure

4.13.1 Acquirer List Group

A table containing a list of all available acquirers.

4.13.1.1 Acquirer Identifier

This element is a unique code identifying the acquirer (or acquirer-processor).
The C-TAP Authority allocates (i.e. assigns and publishes) the applicable (registered) values.

4.13.1.2 Acquirer Label Name

This element gives a textual transcription of the (official) name registered for the acquirer, intended for merchant information (e.g. at acquirer-brand links configuration or consultation).
The C-TAP Authority manages (i.e. collects and publishes) the applicable (registered) values.

4.14 Acquirer Download Request

This is a request from the ECR to the terminal. The ECR asks C-TAP to perform a download of the acquirer (parameter download), identified by the Acquirer Identifier. There is no response message but information reports can be sent.

Table 35 describes the structure of the *AcquirerDownloadRequest* message.

			Acquirer Download	
Data Element	**Tag**	**Format**	**C-**	**R-**
Message Header	E1	GRP	M	
• Protocol ID	10	8i	M	
• Message Type	11	8i	M	
• Terminal ID	12	1..16a	E	
• ECR ID	13	1..16a	M	
• Sequence Number	14	4x	M	
• Encryption Parameters	A0	GRP	C	
Message Body	E2	GRP	M	
• Acquirer Identifier	DF68	8i	M	
Message Footer	E3	GRP	M	
• MDC	1C	8..128x	M	

Table 35: Acquirer Download request message structure

4.14.1 Acquirer Identifier

The ID of the acquirer to perform a download for.

4.15 Acquirer List Pending Messages Request/Response

This is a request from the ECR to the terminal. The ECR asks C-TAP to give a list of acquirers that have pending messages. The use of this list is to choose one acquirer to do a send or delete pending message for.

Table 36 describes the structure of the *AcquirerListPendingMessagesRequest* and *AcquirerListPendingMessagesRequest* messages.

			Acquirer List Pending Messages	
Data Element	**Tag**	**Format**	**C-**	**R-**
Message Header	E1	GRP	M	M
• Protocol ID	10	8i	M	M
• Message Type	11	8i	M	M
• Terminal ID	12	1..16a	E	M
• ECR ID	13	1..16a	M	E
• Sequence Number	14	4x	M	M
• Encryption Parameters	A0	GRP	C	C
Message Body	E2	GRP	M	M
• Acquirer List	E7	GRP		M
○ Table Record	EF	REC 0..x		O
▪ Acquirer Identifier	DF68	8i		M
▪ Acquirer Label Name	DF69	0..16a		M
Message Footer	E3	GRP	M	M
• MDC	1C	8..128x	M	M

Table 36: Acquirer list pending messages request and response message structure

4.15.1 Acquirer List Group

A table containing a list of all acquirers that have pending messages. See section 4.13.1 for more information.

4.16 Send or Delete Pending Messages Request

This is a request from the ECR to the terminal. The ECR asks C-TAP to send or delete pending messages of the acquirer, identified by the Acquirer Identifier. There is no response message but information reports can be sent.

Table 37 describes the structure of the *SendOrDeletePendingMessagesRequest* message.

			Send Or Delete Pending Messages	
Data Element	**Tag**	**Format**	**C-**	**R-**
Message Header	E1	GRP	M	
• Protocol ID	10	8i	M	
• Message Type	11	8i	M	
• Terminal ID	12	1..16a	E	
• ECR ID	13	1..16a	M	
• Sequence Number	14	4x	M	
• Encryption Parameters	A0	GRP	C	
Message Body	E2	GRP	M	
• Pending Messages Operation	C2	1x	M	
• Acquirer Identifier	DF68	8i	M	
Message Footer	E3	GRP	M	
• MDC	1C	8..128x	M	

Table 37: Send or delete pending messages request message structure

4.16.1 Pending Messages Operation

Determines the operation to perform on the pending messages. See Table 38 for possible values.

ID	Description
1	Send pending messages
2	Delete pending messages

Table 38: Possible values for the pending messages operation

4.16.2 Acquirer Identifier

The ID of the acquirer to do send or delete the pending messages for.

4.17 TMS Session Request/Response

Request the terminal to do a TMS session. Currently the response message is empty.

Table 39 describes the structure of the *TmsSessionRequest* and *TmsSessionResponse* messages.

			TMS session	
Data Element	**Tag**	**Format**	**C-**	**R-**
Message Header	E1	GRP	M	M
• Protocol ID	10	8i	M	M
• Message Type	11	8i	M	M
• Terminal ID	12	1..16a	M	E
• ECR ID	13	1..16a	E	M
• Sequence Number	14	4x	M	E
• Encryption Parameters	A0	GRP	C	C
Message Body	E2	GRP	M	M
Message Footer	E3	GRP	M	M
• MDC	1C	8..128x	M	M

Table 39: TMS session request and response messages structure

4.18 Reboot and Clear CTAP Database Request

Request the terminal to clear the C-TAP database and reboot. There is no response message. The terminal will disconnect and reboot.

Table 40 describes the structure of the *TmsSessionRequest* message.

			Reboot and Clear C-TAP database	
Data Element	**Tag**	**Format**	**C-**	**R-**
Message Header	E1	GRP	M	
• Protocol ID	10	8i	M	
• Message Type	11	8i	M	
• Terminal ID	12	1..16a	E	
• ECR ID	13	1..16a	M	
• Sequence Number	14	4x	M	
• Encryption Parameters	A0	GRP	C	
Message Body	E2	GRP	M	
• Reboot and Clear Type	C1	2i	M	
Message Footer	E3	GRP	M	
• MDC	1C	8..128x	M	

Table 40: Clear C-TAP database request message structure

4.18.1 Reboot and Clear Type

Determines what of the C-TAP database is cleared. The possible options are described in Table 41.

ID	Description
0	Just reboot.
1	Partially clear C-TAP database (acquirer data and totals are not deleted).
2	Fully clear the C-TAP database (only the totals are not deleted).
3	Delete the C-TAP database and recreated it empty.

Table 41: The reboot and clear C-TAP database options

4.19 Version Information Request/Response

With the *VersionInformationRequest* message, it is possible to request the version information of the terminal applications. Also additional information can be returned like disk space and database sizes as name/value/unit triplets.

Table 42 describes the structure of the *VersionInformationRequest* and *VersionInformationResponse* messages.

			Version Info	
Data Element	**Tag**	**Format**	**C-**	**R-**
Message Header	E1	GRP	M	M
• Protocol ID	10	8i	M	M
• Message Type	11	8i	M	M
• Terminal ID	12	1..16a	M	E
• ECR ID	13	1..16a		M
• Sequence Number	14	4x	M	M
• Encryption Parameters	A0	GRP	O	O
Message Body	E2	GRP	M	M
• Application ID	80	2x	M	E
• Terminal Logical ID	81	1..16a		M
• Terminal Serial Number	82	1..32a		M
• Version Number (Major, Minor, Build, Revision)	83	8i		M
• Version String	84	1..64a		M
• Extra Information	A7	TBL		O
○ Table Record	EF	REC 1..n		M
▪ Name	85	1..16a		M
▪ Value	86	1..64a		M
▪ Unit	87	2x		M
• Result Code	16	4x		M
Message Footer	E3	GRP	M	M
• MDC	1C	8..128x	M	M

Table 42: Version information request and response message structure

4.19.1 Application ID

The application ID is the terminal application/system of which the version information is desired.

Name	ID	Description
SDK	0	The terminal SDK and system.
RDM	1	The terminal RDM application (Remote Device Management).
C-TAP	2	The terminal C-TAP application (C-TAP payment application).
ECR	3	The terminal ECR application (Electronic Cash Register interface).
TSC	4	The terminal TSC application (Secure Component).
MDB	5	The terminal MDB application (vending machine interface).
Custom	6	The terminal custom payment application.

Table 43: Application IDs

4.19.2 Terminal Logical ID

This is the logical ID of the terminal.

4.19.3 Terminal Serial Number

This is the serial number of the terminal (also called physical ID).

4.19.4 Version Number

The version number as BCD number. Each byte (2 digits) stands for a part of the version number (major, minor, build, revision). The terminal applications only supports major and minor number, the build and revision will always be 0.
When there is no version information (result code "*NotAvailable*"), the version number will all be all zeros.

4.19.5 Version String

This is the version string. For example "RDM v1.51 - (C) 2015-2017 Chess pT [E84D]".
When there is no version information (result code "*NotAvailable*), the version string will be empty.

4.19.6 Extra Information

The extra information is an optional table containing name/value/unit triplets. Some applications return here extra information like disk space, database sizes, etc.

The unit field is divided in two bytes. The first signed byte indicates the unit prefix like milli or kilo and determines the 10^x multiplication factor of the unit (see Table 44 for possible values). The second byte indicates the base unit like bytes or seconds (see Table 45 for possible values).

For example if the unit (2nd byte) is 2 (seconds) and the unit prefix (1st byte) is -3 (milli) then the actual unit is milliseconds.

Name	ID	Description
Pico	-12	Pico unit prefix (10^{-12})
Nano	-9	Nano unit prefix (10^{-9})
Micro	-6	Micro unit prefix (10^{-6})
Milli	-3	Milli unit prefix (10^{-3})
Centi	-2	Centi unit prefix (10^{-2})
Deci	-1	Deci unit prefix (10^{-1})
None	0	No unit prefix.
Deca	1	Deca unit prefix (10^{1})
Hecto	2	Hecto unit prefix (10^{2})
Kilo	3	Kilo unit prefix (10^{3})
Mega	6	Mega unit prefix (10^{6})
Giga	9	Giga unit prefix (10^{9})
Tera	12	Tera unit prefix (10^{12})
Peta	15	Peta unit prefix (10^{15})

Table 44: Unit Prefixes byte (first unit byte)

Name	ID	Description
None	0	No unit specified.
Bytes	1	Bytes.
Seconds	2	Seconds.
Bps	3	Bits per second.

Table 45: Unit byte (second unit byte)

4.19.7 Result Code

The result code indicates the result of the request (success or the reason for failure). When there was version information (wrong application ID or the application is not installed), "*NotAvailable*" is returned.

A Implementation Checklist

Below a checklist for ECR implementations:

- The sequence number of messages sent from the ECR are independent from the sequence number of messages sent from the terminal. They both have their own sequence number counter.
- Since the sequence number is a 16 bit integer, after sequence number 65535 the number wraps back to 0. When checking the current received number with the last received, this should be taken into account.
- An ECR should not only respond to keep-alive requests with a keep-alive response but should also check if there are keep-alive requests received.
- Since sending commands (transaction request, etc.) is independent from responding to keep-alive requests the code that fills the sequence number and sends the message must be an atomic operation (not interrupted by another send). Else the sequence number of message 2 can be generated and sent after the sequence number of message 1 is generated and before message 1 is sent resulting in the wrong order of sequence numbers. In a multi-threaded implementation this can be synchronised using mutexes. Alternatively message can be queued and sent from a queue ensuring generating the sequence number and sending the message always occur on the same thread.

B Enumerations

B.1 Message Types

The message type indicates what message is stored in the body of a message. Table 46 describes the defined messages with their code, the direction (from terminal to ECR or from ECR to terminal) and from which version (Protocol ID) the message is implemented.

Name	ID (dec)	Description	Source	Dest	Version
Hello Request	16	Request a connection with the cash register.	Terminal	ECR	1
Hello Response	17	Result of the connection request.	ECR	Terminal	1
Keep-alive Request	18	Notification of status and keep-alive.	Terminal	ECR	1
Keep-alive Response	19	Result of the notification.	ECR	Terminal	1
Bye Request	20	Request to terminate the connection.	Both	Both	1
Bye Response	21	Result of terminate connection request.	Both	Both	1
Acquirer List Request	30	Request a list of all available acquirers.	ECR	Terminal	1
Acquirer List Response	31	Result of the acquirer list request.	Terminal	ECR	1
Acquirer List Pending Messages Request	32	Requests a list with acquirers that have pending messages.	ECR	Terminal	1
Acquirer List Pending Messages Response	33	Result of the acquirer list request.	Terminal	ECR	1
Send or Delete Pending Messages Request	34	Request sending or deleting pending messages for the given acquirer.	ECR	Terminal	1
Acquirer Download Request	36	Request an acquirer download for the given acquirer.	ECR	Terminal	1
Security Scheme List Request	38	Request a list of all available security schemes.	ECR	Terminal	1
Security Scheme List Response	39	Result of the security scheme list request.	Terminal	ECR	1
Security Scheme Download Request	40	Request a keyload for the given acquirer.	ECR	Terminal	1
Print Info Request	46	Request printing information.	ECR	Terminal	1
Print Info Response	47	Result of the print information request.	Terminal	ECR	1
Transaction Request	48	Request to perform a transaction.	ECR	Terminal	1

Name	ID (dec)	Description	Source	Dest	Version
Transaction Response	49	Result of the transaction.	Terminal	ECR	1
Totals Request	50	Request an overview of the counters (print on terminal or send to the cash register).	ECR	Terminal	1
Totals Response	51	Result of the totals request.	Terminal	ECR	1
Last Transaction Ticket Request	52	Request the last transaction ticket (print on terminal or send to the cash register).	ECR	Terminal	1
Last Transaction Ticket Response	53	Result of the last transaction ticket request.	Terminal	ECR	1
Cancel Request	54	Request the cancellation of an on-going operation.	ECR	Terminal	1
Cancel Response	55	Result of the cancellation request.	Terminal	ECR	1
Last Transaction Status Request	56	Request the status of the last transaction.	ECR	Terminal	1
Last Transaction Status Response	57	Result of the last transaction status request.	Terminal	ECR	1
Print Confirmation Request	64	Print confirmation from the cash register to the terminal when a card holder ticket must be printed on the cash register.	ECR	Terminal	
Print Confirmation Response	65	Result of the print confirmation request.	Terminal	ECR	
Print Request	66	Request to print some data (e.g. the cash register ticket) on the terminal printer. The cash register does not need a printer then.	ECR	Terminal	
Print Response	67	Result of the print request.	Terminal	ECR	
Intermediate Result Request	68	Report the intermediate result.	Terminal	ECR	1
Intermediate Result Response	69	Result of the intermediate result report (continue/abort transaction).	ECR	Terminal	1
Information Report	80	Report the progress of a transaction and other information like merchant messages.	Terminal	ECR	1
Settings Request	82	Change one or more settings in the terminal.	ECR	Terminal	
Settings Response	83	Result of the change settings request.	Terminal	ECR	
Version Information Request	90	Request the version of the terminal software.	ECR	Terminal	1
Version Information Response	91	Result of the version request.	Terminal	ECR	1

Name	ID (dec)	Description	Source	Dest	Version
TMS Request	92	Start a TMS session on the terminal.	ECR	Terminal	1
TMS Response	93	Result of the TMS request.	Terminal	ECR	1
Reboot and Clear C-TAP Database Request	100	Reboot and clear the C-TAP database on the terminal.	ECR	Terminal	1

Table 46: Message types

B.2 Result Codes

Each response message has a result code. Table 47 describes the result codes that could occur.

Name	ID (decimal)	Description
ResultOK	0	All OK, no problems.
PrinterError	1	Printer problem.
BatteryError	2	Battery problem (battery empty).
NetworkError	3	There is no network connection (e.g. Ethernet cable unplugged, GPRS modem has low signal, no WiFi signal, etc.).
RdmBusy	4	RDM is busy. Try again later.
TdrFull	5	There is no space for TDRs.
Cancelled	10	The operation is cancelled.
CancelFailed	11	Could not cancel operation.
Rejected	12	The request was rejected.
NotAvailable	13	The requested data was not available.
LastTicketNotPrinted	14	The last ticket was not yet printed.
NoPendingPrintRequest	15	Send in response to a print confirmation when there was no pending print request for a card holder, merchant or journal ticket.
IncompatibleVersion	16	A message was received that is incompatible (e.g. the protocol version is not supported).
CardNotRemoved	17	The card was not removed.
ParameterMissing	18	One of the required parameters is missing.
ParameterInvalid	19	One of the parameters was invalid (wrong length, invalid value).
CtapBlocked	20	The terminal is blocked for payments.
CtapInvalidAmount	21	The payment amount was not supported (e.g. too big).
CtapInvalidCurrency	22	The selected currency was not supported.
CtapCustomerAborted	23	The customer aborted.
CtapMerchantAborted	24	The merchant aborted.
CtapCardRemoved	25	The card was removed.
CtapCardNotSupported	26	The card was not supported.
CtapCardInvalid	27	The card was invalid.
CtapCardProblem	28	Unknown card problem.
CtapBusy	29	CTAP is busy. Try again later.
CtapError	30	Unspecified CTAP error.

Table 47: Result codes

B.3 Transaction Stage

The transaction stage is used to indicate the current transaction stage in an “Information Report” message. The possible stages are described in Table 48.

Name	ID (decimal)	Description
None	0	The terminal is not in a transaction.
WaitingForCard	1	The terminal is waiting for a card.
WaitingForPin	2	The terminal is waiting for a PIN code.
WaitingForTransaction	3	The terminal is performing a transaction.
Finished	4	The terminal is finished with a transaction.
WaitingForTipInput	5	The terminal is waiting for a card holder TIP input.
WaitingForConfirmationService	6	The terminal is waiting for a card holder service confirmation.
WaitingForConfirmationAmount	7	The terminal is waiting for a card holder amount confirmation.
WaitingForConfirmationServiceAndAmount	8	The terminal is waiting for a card holder service and amount confirmation.
WaitingForCardRemoval	9	The terminal is waiting till the card is removed.
WaitingForLastTransactionResult	10	The terminal is displaying the last transaction information on the terminal.
WaitingForApplicationSelection	11	The terminal is waiting for the application selection (In case a card contains multiple applications e.g. selection between Visa debit or credit).
CardDetected	12	The terminal detected a card (CLess) or a card was inserted (ICC) or swiped (MagStripe).
WaitingForIntermediateResult	13	The terminal is waiting for an intermediate result from the cash register (in case of deferred payment transaction or transaction with intermediate result).
CardRemoved	14	The card was removed.

Table 48: Transaction stages

C CRC-32 Algorithm

The default modification detection algorithm is CRC-32. See the following example written in C:

```
/*-
 *  COPYRIGHT (C) 1986 Gary S. Brown.  You may use this program, or
 *  code or tables extracted from it, as desired without restriction.
 *
 *  First, the polynomial itself and its table of feedback terms.  The
 *  polynomial is
 *  X^32+X^26+X^23+X^22+X^16+X^12+X^11+X^10+X^8+X^7+X^5+X^4+X^2+X^1+X^0
 *
 *  Note that we take it "backwards" and put the highest-order term in
 *  the lowest-order bit.  The X^32 term is "implied"; the LSB is the
 *  X^31 term, etc.  The X^0 term (usually shown as "+1") results in
 *  the MSB being 1
 *
 *  Note that the usual hardware shift register implementation, which
 *  is what we're using (we're merely optimizing it by doing eight-bit
 *  chunks at a time) shifts bits into the lowest-order term.  In our
 *  implementation, that means shifting towards the right.  Why do we
 *  do it this way?  Because the calculated CRC must be transmitted in
 *  order from highest-order term to lowest-order term.  UARTs transmit
 *  characters in order from LSB to MSB.  By storing the CRC this way
 *  we hand it to the UART in the order low-byte to high-byte; the UART
 *  sends each low-bit to hight-bit; and the result is transmission bit
 *  by bit from highest- to lowest-order term without requiring any bit
 *  shuffling on our part.  Reception works similarly
 *
 *  The feedback terms table consists of 256, 32-bit entries.  Notes
 *
 *      The table can be generated at runtime if desired; code to do so
 *      is shown later.  It might not be obvious, but the feedback
 *      terms simply represent the results of eight shift/xor opera
 *      tions for all combinations of data and CRC register values
 *
 *      The values must be right-shifted by eight bits by the "updcrc
 *      logic; the shift must be unsigned (bring in zeroes).  On some
 *      hardware you could probably optimize the shift in assembler by
 *      using byte-swap instructions
 *      polynomial $edb88320
 *
 *
 * CRC32 code derived from work by Gary S. Brown.
 */

#include <sys/param.h>
#include <sys/systm.h>

static uint32_t crc32_tab[] =
{
  0x00000000, 0x77073096, 0xee0e612c, 0x990951ba, 0x076dc419, 0x706af48f,
  0xe963a535, 0x9e6495a3, 0x0edb8832, 0x79dcb8a4, 0xe0d5e91e, 0x97d2d988,
  0x09b64c2b, 0x7eb17cbd, 0xe7b82d07, 0x90bf1d91, 0x1db71064, 0x6ab020f2,
  0xf3b97148, 0x84be41de, 0x1adad47d, 0x6ddde4eb, 0xf4d4b551, 0x83d385c7,
  0x136c9856, 0x646ba8c0, 0xfd62f97a, 0x8a65c9ec, 0x14015c4f, 0x63066cd9,
  0xfa0f3d63, 0x8d080df5, 0x3b6e20c8, 0x4c69105e, 0xd56041e4, 0xa2677172,
  0x3c03e4d1, 0x4b04d447, 0xd20d85fd, 0xa50ab56b, 0x35b5a8fa, 0x42b2986c,
  0xdbbbc9d6, 0xacbcf940, 0x32d86ce3, 0x45df5c75, 0xdcd60dcf, 0xabd13d59,
  0x26d930ac, 0x51de003a, 0xc8d75180, 0xbfd06116, 0x21b4f4b5, 0x56b3c423,
  0xcfba9599, 0xb8bda50f, 0x2802b89e, 0x5f058808, 0xc60cd9b2, 0xb10be924,
  0x2f6f7c87, 0x58684c11, 0xc1611dab, 0xb6662d3d, 0x76dc4190, 0x01db7106,
  0x98d220bc, 0xefd5102a, 0x71b18589, 0x06b6b51f, 0x9fbfe4a5, 0xe8b8d433,
  0x7807c9a2, 0x0f00f934, 0x9609a88e, 0xe10e9818, 0x7f6a0dbb, 0x086d3d2d,
  0x91646c97, 0xe6635c01, 0x6b6b51f4, 0x1c6c6162, 0x856530d8, 0xf262004e,
  0x6c0695ed, 0x1b01a57b, 0x8208f4c1, 0xf50fc457, 0x65b0d9c6, 0x12b7e950,
  0x8bbeb8ea, 0xfcb9887c, 0x62dd1ddf, 0x15da2d49, 0x8cd37cf3, 0xfbd44c65,
  0x4db26158, 0x3ab551ce, 0xa3bc0074, 0xd4bb30e2, 0x4adfa541, 0x3dd895d7,
  0xa4d1c46d, 0xd3d6f4fb, 0x4369e96a, 0x346ed9fc, 0xad678846, 0xda60b8d0,
  0x44042d73, 0x33031de5, 0xaa0a4c5f, 0xdd0d7cc9, 0x5005713c, 0x270241aa,
  0xbe0b1010, 0xc90c2086, 0x5768b525, 0x206f85b3, 0xb966d409, 0xce61e49f,
```

```
  x5edef90e, 0x29d9c998, 0xb0d09822, 0xc7d7a8b4, 0x59b33d17, 0x2eb40d81,
  0xb7bd5c3b, 0xc0ba6cad, 0xedb88320, 0x9abfb3b6, 0x03b6e20c, 0x74b1d29a,
  0xead54739, 0x9dd277af, 0x04db2615, 0x73dc1683, 0xe3630b12, 0x94643b84,
  0x0d6d6a3e, 0x7a6a5aa8, 0xe40ecf0b, 0x9309ff9d, 0x0a00ae27, 0x7d079eb1,
  0xf00f9344, 0x8708a3d2, 0x1e01f268, 0x6906c2fe, 0xf762575d, 0x806567cb,
  0x196c3671, 0x6e6b06e7, 0xfed41b76, 0x89d32be0, 0x10da7a5a, 0x67dd4acc,
  0xf9b9df6f, 0x8ebeeff9, 0x17b7be43, 0x60b08ed5, 0xd6d6a3e8, 0xa1d1937e,
  0x38d8c2c4, 0x4fdff252, 0xd1bb67f1, 0xa6bc5767, 0x3fb506dd, 0x48b2364b,
  0xd80d2bda, 0xaf0a1b4c, 0x36034af6, 0x41047a60, 0xdf60efc3, 0xa867df55,
  0x316e8eef, 0x4669be79, 0xcb61b38c, 0xbc66831a, 0x256fd2a0, 0x5268e236,
  0xcc0c7795, 0xbb0b4703, 0x220216b9, 0x5505262f, 0xc5ba3bbe, 0xb2bd0b28,
  0x2bb45a92, 0x5cb36a04, 0xc2d7ffa7, 0xb5d0cf31, 0x2cd99e8b, 0x5bdeae1d,
  0x9b64c2b0, 0xec63f226, 0x756aa39c, 0x026d930a, 0x9c0906a9, 0xeb0e363f,
  0x72076785, 0x05005713, 0x95bf4a82, 0xe2b87a14, 0x7bb12bae, 0x0cb61b38,
  0x92d28e9b, 0xe5d5be0d, 0x7cdcefb7, 0x0bdbdf21, 0x86d3d2d4, 0xf1d4e242,
  0x68ddb3f8, 0x1fda836e, 0x81be16cd, 0xf6b9265b, 0x6fb077e1, 0x18b74777,
  0x88085ae6, 0xff0f6a70, 0x66063bca, 0x11010b5c, 0x8f659eff, 0xf862ae69,
  0x616bffd3, 0x166ccf45, 0xa00ae278, 0xd70dd2ee, 0x4e048354, 0x3903b3c2,
  0xa7672661, 0xd06016f7, 0x4969474d, 0x3e6e77db, 0xaed16a4a, 0xd9d65adc,
  0x40df0b66, 0x37d83bf0, 0xa9bcae53, 0xdebb9ec5, 0x47b2cf7f, 0x30b5ffe9,
  0xbdbdf21c, 0xcabac28a, 0x53b39330, 0x24b4a3a6, 0xbad03605, 0xcdd70693,
  0x54de5729, 0x23d967bf, 0xb3667a2e, 0xc4614ab8, 0x5d681b02, 0x2a6f2b94,
  0xb40bbe37, 0xc30c8ea1, 0x5a05df1b, 0x2d02ef8d
};

uint32_t crc32(uint32_t crc, const void *buf, size_t size)
{
  const uint8_t *p;

  p = buf;
  crc = crc ^ ~0U;

  while (size--)
  {
    crc = crc32_tab[(crc ^ *p++) & 0xFF] ^ (crc >> 8);
  }

  return crc ^ ~0U;
}
```

This example code was located at http://www.opensource.apple.com/source/xnu/xnu-1456.1.26/bsd/libkern/crc32.c

D Luhn Check Algorithm

The “Luhn mod 10 plus 1” algorithm is used for checking the token (see section 4.5.2.11) which is a variant of the “Luhn mod 10” algorithm (see [6]). Note that the check is performed on the binary value of a digit, not the ASCII value.

The following sections provide a C++ and C# example to check if a numeric string fulfils the “Luhn mod 10” and “Luhn mod 10 plus 1” check.

D.1 Luhn Check Algorithm (C++)

```
//! Perform Luhn check (modulus 10 algorithm) on a numeric string.
/*!
 * https://en.wikipedia.org/wiki/Luhn_algorithm
 *
 * 1) From the rightmost digit, which is the check digit, and moving left,
 *    double the value of every second digit.
 *    If the result of this doubling operation is greater than 9
 *    (e.g., 8 × 2 = 16), then add the digits of the product
 *    (e.g., 16: 1 + 6 = 7, 18: 1 + 8 = 9) or alternatively subtract 9
 *    from the product (e.g., 16: 16 - 9 = 7, 18: 18 - 9 = 9).
 * 2) Take the sum of all the digits.
 * 3) If the total modulo 10 is equal to 0 (if the total ends in zero)
 *    then the number is valid according to the Luhn formula,
 *    else it is not valid.
 *
 * Example (the rightmost number is the check digit):
 *
 * Account number     7   9   9   2   7   3   9   8   7   1   3
 * Double every other 7   18  9   4   7   6   9   16  7   2   3
 * Sum digits         7   9   9   4   7   6   9   7   7   2   3
 *
 * The sum = 70. Modulo 10 of 70 is 0 thus the check succeeds.
 *
 *
 * \param[in] code  The code to check.
 *
 * \return          True when the Luhn check is OK.
 */
bool LuhnCheckMod10(const std::string& code)
{
  // Exit if the string length is less than 2.
  if (code.length() < 2) return false;

  // Iterate each digit
  // (from back to front as doubling the even position digits is from the right).
  int digitCounter = 0;  // Counter used to double the even position digits.
  int sum = 0;           // The sum of the "sum" digits.
  std::string::const_reverse_iterator end = code.rend();
  for (std::string::const_reverse_iterator it = code.rbegin(); it != end; ++it)
  {
    // Get the numeric value of the ASCII digit.
    int currentDigit = *it - 0x30;

    // Exit if not numeric.
    if (currentDigit > 9) return false;

    // Double if even position digit (from the right).
    if ( (digitCounter++ % 2) != 0) currentDigit *= 2;

    // Sum the digit.
    sum += (currentDigit / 10) + (currentDigit % 10);
  }

  // When the sum ends with '0' (divisible by 10), then valid code.
```

```
  return sum % 10 == 0;
}


//! Perform +1 modified Luhn check (modulus 10 plus 1) on a numeric string.
/*!
 * This is a modified Luhn check where the check digit is incremented by 1.
 *
 * \param[in] code  The code to check.
 *
 * \return          True when the Luhn check is OK.
 */
bool LuhnCheckMod10Plus1(const std::string& code)
{
  // Since the check digit was increased, decrement the check digit (last digit)
  // before passing it to the regular Luhn check.
  // When check digit is zero (thus subtracting results in -1),
  // the modified check digit becomes 9.
  std::string decrementedCode = code;
  int index = decrementedCode.length() - 1;
  char checkDigit = decrementedCode[index];
  decrementedCode[index] = checkDigit != '0' ? checkDigit - 1 : '9';
  return LuhnCheckMod10(decrementedCode);
}
```

D.2 Luhn Check Algorithm (C#)

```
/// <summary>
/// Class providing Luhn check functionality.
/// https://en.wikipedia.org/wiki/Luhn_algorithm
/// </summary>
public static class LuhnCheck
{
  /// <summary>
  /// Perform Luhn check (modulus 10 algorithm) on a numeric string.
  ///
  /// 1) From the rightmost digit, which is the check digit, and moving left,
  ///    double the value of every second digit.
  ///    If the result of this doubling operation is greater than 9
  ///    (e.g., 8 × 2 = 16), then add the digits of the product
  ///    (e.g., 16: 1 + 6 = 7, 18: 1 + 8 = 9) or alternatively subtract 9
  ///    from the product (e.g., 16: 16 - 9 = 7, 18: 18 - 9 = 9).
  /// 2) Take the sum of all the digits.
  /// 3) If the total modulo 10 is equal to 0 (if the total ends in zero)
  ///    then the number is valid according to the Luhn formula,
  ///    else it is not valid.
  ///
  /// Example (the rightmost number is the check digit):
  ///
  /// Account number      7   9   9   2   7   3   9   8   7   1   3
  /// Double every other  7   18  9   4   7   6   9   16  7   2   3
  /// Sum digits          7   9   9   4   7   6   9   7   7   2   3
  ///
  /// The sum = 70. Modulo 10 of 70 is 0 thus the check succeeds.
  /// </summary>
  /// <param name="code">The code to check.</param>
  /// <returns>True when the Luhn check is OK.</returns>
  public static bool LuhnCheckMod10(string code)
  {
    // Exit if the string length is less than 2.
    if (code.Length < 2) return false;

    return code
      .Where(c => Char.IsDigit(c))           // Select only digits.
      .Reverse()                             // Reverse the sequence since we
                                             // count from the right for doubling
                                             // the even position digits.
      .Select((c, p) => c - '0' << (p & 1))  // Double the even position digits.
      .Select(n => n > 9 ? n - 9 : n)        // When digit became larger than 9,
```

```
                                                    // subtract it from 9.
      .Sum()                                        // Sum the digits.
      % 10 == 0;                                    // Code correct when divisible by 10
                                                    // (sum ends with '0').
  }


  /// <summary>
  /// Perform +1 modified Luhn check (modulus 10 algorithm plus 1)
  /// on a numeric string.
  /// This is a modified Luhn check where the check digit is incremented by 1.
  /// </summary>
  /// <param name="code">The code to check.</param>
  /// <returns>True when the Luhn check is OK.</returns>
  public static bool LuhnCheckMod10Plus1(string code)
  {
    // Exit if one or more digits are not numeric.
    foreach (char c in code)
    {
      if (!Char.IsDigit(c)) return false;
    }

    // Since the check digit was increased, decrement the check digit (last digit)
    // before passing it to the regular Luhn check.
    // When check digit is zero (thus subtracting results in -1),
    // the modified check digit becomes 9.
    string checkDigit = code.Substring(code.Length - 1);
    string decrementedCode = code.Substring(0, code.Length - 1);
    decrementedCode += checkDigit != "0" ?
                       (Int32.Parse(checkDigit) - 1).ToString() : "9";
    return LuhnCheckMod10(decrementedCode);
  }
}
```

E Example Messages

The following sections show some examples of TLV messages. The bytes of the message have the following colour/formatting codes:

- *Message length*
- **Magic header**
- TLV Tag
- *TLV length*
- Primitive TLV data

E.1 Hello Request

Sample *HelloRequest* message data:

```
00 00 00 4A 50 34 59 2D   45 43 52 21 F0 40 E1 1E
11 04 00 00 00 16 10 04   00 00 00 01 12 0A 31 30
30 30 30 35 38 37 31 39   13 00 14 02 00 01 E2 16
A4 04 49 02 00 00 A1 00   A2 0C EF 04 18 02 6E 6C
EF 04 18 02 65 6E E3 06   1C 04 67 81 B6 9D
```

Tag	Tag Name	Format	Content	Length	Encoding
	Msg length	8x	74	-	*00 00 00 4A*
	Magic	8a	"P4Y-ECR!"	-	**50 34 59 2D 45 43 52 21**
F0	Root	GRP	…	64	F0 *40*
E1	Header	GRP	…	30	E1 *1E*
11	Message type	8i	16	4	11 *04* 00 00 00 16
10	Protocol ID	8i	1	4	10 *04* 00 00 00 01
12	Terminal ID	1..16a	"1000058719"	10	12 *0A* 31 30 30 30 30 35 38 37 31 39
13	ECR ID	1..16a	""	0	13 *00*
14	Sequence Nr	4x	1	2	14 *02* 00 01
E2	Body	GRP	…	22	E2 *16*
A4	Connection Parameters	GRP	…	4	A4 *04*
49	Keep-alive interval	4i	0	2	49 *02* 00 00
A1	Communication Parameters	GRP	…	0	A1 *00*
A2	Supported Languages	TBL	…	12	A2 *0C*
EF		REC	…	4	EF *04*
18	Language	2a	"nl"	2	18 *02* 6E 6C
EF		REC	…	4	EF *04*
18	Language	2a	"en"	2	18 *02* 65 6E
E3	Footer	GRP	…	6	E3 *06*
1C	MDC	8..128x	0x6781B69D	4	1C *04* 67 81 B6 9D

Table 49: Example *HelloRequest* message dissected

E.2 Hello Response

Sample *HelloResponse* message data:

```
00 00 00 45 50 34 59 2D   45 43 52 21 F0 3B E1 25
10 04 00 00 00 01 11 04   00 00 00 17 12 0A 31 30
30 30 30 35 38 37 31 39   13 07 54 65 73 74 45 43
52 14 02 00 01 E2 0A 16   02 00 00 A4 04 49 02 00
00 E3 06 1C 04 75 59 87   BE
```

Tag	Tag Name	Format	Content	Length	Encoding
	Msg length	8x	69	-	*00 00 00 45*
	Magic	8a	"P4Y-ECR!"	-	**50 34 59 2D 45 43 52 21**
F0	Root	GRP	…	59	F0 *3B*
E1	Header	GRP	…	37	E1 *25*
10	Protocol ID	8i	1	4	10 *04* 00 00 00 01
11	Message type	8i	17	4	11 *04* 00 00 00 17
12	Terminal ID	1..16a	"1000058719"	10	12 *0A* 31 30 30 30 30 35 38 37 31 39
13	ECR ID	1..16a	"TestECR"	7	13 *07* 54 65 73 74 45 43 52
14	Sequence Nr	4x	1	2	14 *02* 00 01
E2	Body	GRP	…	10	E2 *0A*
16	Result Code	4x	0	2	16 *02* 00 00
A4	Connection Parameters	GRP	…	4	A4 *04*
49	Keep-alive interval	4i	0	2	49 *02* 00 00
E3	Footer	GRP	…	6	E3 *06*
1C	MDC	8..128x	0x755987BE	4	1C *04* 75 59 87 BE

Table 50: Example *HelloResponse* dissected

E.3 Transaction Request

Sample *TransactionRequest* message data:

```
00 00 00 5E 50 34 59 2D   45 43 52 21 F0 54 E1 25
10 04 00 00 00 01 11 04   00 00 00 48 12 0A 31 30
30 30 30 35 38 37 31 39   13 07 54 65 73 74 45 43
52 14 02 00 02 E2 23 A3   12 50 04 00 00 12 34 51
02 09 78 52 01 02 55 00   54 01 01 A5 03 4E 01 02
1A 08 00 00 00 00 00 00   00 00 E3 06 1C 04 DE C2
19 BD
```

E.4 Transaction Response

Sample *TransactionResponse* message data:

```
00 00 00 B7 50 34 59 2D   45 43 52 21 F0 81 AC E1
25 11 04 00 00 00 49 10   04 00 00 00 01 12 0A 31
30 30 30 30 35 38 37 31   39 13 07 54 65 73 74 45
43 52 14 02 00 02 E2 7B   19 02 00 1F 56 01 00 1A
08 00 00 00 00 00 00 00   00 16 01 00 A6 65 EF 22
4A 01 02 4B 00 4C 19 44   75 6D 6D 79 20 63 61 72
64 20 68 6F 6C 64 65 72   20 74 69 63 6B 65 74 2E
4D 00 EF 1F 4A 01 01 4B   00 4C 16 44 75 6D 6D 79
20 6D 65 72 63 68 61 6E   74 20 74 69 63 6B 65 74
```

```
2E 4D 00 EF 1E 4A 01 03   4B 00 4C 15 44 75 6D 6D
79 20 6A 6F 75 72 6E 61   6C 20 74 69 63 6B 65 74
2E 4D 00 E3 06 1C 04 31   16 EA 70
```

E.5 Bye Request

Sample *ByeRequest* message data:

```
00 00 00 3E 50 34 59 2D   45 43 52 21 F0 34 E1 25
10 04 00 00 00 01 11 04   00 00 00 20 12 0A 31 30
30 30 30 35 38 37 31 39   13 07 54 65 73 74 45 43
52 14 02 00 03 E2 03 17   01 01 E3 06 1C 04 88 34
6D 30
```

E.6 Bye Response

Sample *ByeResponse* message data:

```
00 00 00 42 50 34 59 2D   45 43 52 21 F0 38 E1 25
11 04 00 00 00 21 10 04   00 00 00 01 12 0A 31 30
30 30 30 35 38 37 31 39   13 07 54 65 73 74 45 43
52 14 02 00 03 E2 07 16   02 00 00 17 01 01 E3 06
1C 04 A3 DF 63 A3
```